校车管理系统

综合项目报告与技术文档

Flutter 跨平台三端课程原型

课程	软件开发环境与工具
仓库	BUS-FLUTTER
工程	school_bus_app
主要技术	Flutter / Dart / Riverpod / GoRouter
课程交付计划	2026 年 4 月 16 日至 6 月 25 日
文档整合日期	2026 年 10 月 10 日
文档版本	综合整合版 1.0

本文统一收录项目计划、需求、设计、源码说明、测试、配置管理、
工具使用证据和后续技术方案；当前实现与历史记录、规划能力分开表述。

Overleaf 主文件：`main.tex`　编译器：XeLaTeX

摘要与阅读说明

校车管理系统面向学生、教职工、司机及校园运营管理人员，采用一个 Flutter 工程提供普通用户、司机、后台管理三类入口。当前应用使用内置 Mock 数据和 Riverpod 内存状态，能够演示预约选座、模拟支付、取消扣分、司机任务与模拟定位、车辆司机维护、调度建议确认、位置展示和统计看板。

本文不是把原文件逐份附在末尾，而是将相同内容去重，按软件生命周期重新组织，并保留原有 UML 图、计划截图、后台测试截图及可编辑建模证据。所有正文集中于本文件；辅助文件不构成另一套报告。

事实层级：

- 当前实现：以 `pubspec.yaml`、`lib/`、`test/`、CI 工作流及本次实际验证为依据。
- 历史记录：原计划、测试报告、截图中的日期、费用、通过结论仅代表原材料记载，不等于本次重新执行。
- 扩展方案：REST、WebSocket、Supabase 客户端、真实支付、GPS、FCM、地图服务、OpenAI 等尚未接入的能力单独列入规划章节。

导入 Overleaf：选择 New Project → Upload Project，上传 `overleaf.zip`；将主文件设置为 `main.tex`，编译器设置为 XeLaTeX，然后 Recompile。中文排版采用 TeX Live 自带 Fandol 字体，不要求上传系统字体；代码使用普通 Verbatim，不依赖 minted、Python 高亮或 shell-escape。

目录

摘要与阅读说明 **i**

第一章 项目概述与实现范围 **1**

1.1 背景、目标和角色 1

1.2 当前实现与生产能力的区别 1

第二章 项目计划与管理 **3**

2.1 计划口径与时间差异 3

2.2 课程交付工作分解 3

2.3 原 Project 计划与关键路径 4

2.4 资源和成本 5

2.5 沟通、质量和风险管理 7

第三章 需求规格与验收 **9**

3.1 业务需求和用例 9

3.2 功能需求 10

3.3 业务规则与状态语义 11

3.4 非功能需求与验证方式 12

3.5 需求追踪与验收标准 12

第四章 总体架构与详细设计 **14**

4.1 真实依赖与运行入口 14

4.2 模块边界和数据流 15

4.3 核心数据字典 15

4.4 枚举与状态转换 16

4.5 乘客流程 17

4.6 司机任务与定位流程 17
4.7 后台管理、智能调度和分析 18

第五章 UML 建模与图文证据 19
5.1 模型文件和适用范围 19
5.2 用例与领域模型 20
5.3 生命周期、交互与部署 24
5.4 StarUML 设计过程 30

第六章 平台工程与 Windows 宿主设计 39
6.1 平台边界 39
6.2 Windows 入口、窗口和生命周期 39
6.3 构建、容错与扩展 40

第七章 源码使用、运行与演示 41
7.1 开发命令与账号 41
7.2 高价值源码索引 41
7.3 演示顺序 42

第八章 测试计划、用例与报告 43
8.1 测试目标、范围和策略 43
8.2 乘客用例 43
8.3 司机专项用例和测试数据 44
8.4 后台与负向业务用例 45
8.5 可复现的测试代码和命令 46
8.6 本次执行结果 46
8.7 历史测试结论与证据边界 47
8.8 剩余测试风险 51

第九章 配置管理、CI/CD 与文档交付 52
9.1 配置项与版本基线 52
9.2 CI/CD 的实际执行顺序 53
9.3 预备 Supabase SQL 与初始化顺序 53
9.4 单一文档打包与维护 54

第十章 后续全栈技术方案 **55**

10.1 保留的原始选型及其状态 55

10.2 目标 REST API 56

10.3 实时位置、推送与 ETA 56

10.4 支付、数据一致性与调度演进 57

第十一章 工具证据、整合追踪与总结 **58**

11.1 课程工具证据索引 58

11.2 原材料合并与更正记录 58

11.3 最终交付与结论 60

插图

2.1 原 Project 工作分解与任务表：历史计划截图 5
2.2 原 Project 甘特图、依赖和资源分配：历史计划截图 5
2.3 原计划按周现金流：保留原预算分布，不作为真实财务记录 7

5.1 系统总览用例图：普通用户、司机与管理员 20
5.2 乘客端用例详图：查询、选座、支付、取消与通知 21
5.3 历史核心领域类图：当前字段和状态以第四章修订说明为准 22
5.4 历史枚举与业务类型图：包含早期方案的状态命名 23
5.5 预约状态机设计意图：当前预约状态与支付状态分开存储 24
5.6 车次状态机设计：scheduled、running、completed、cancelled 24
5.7 司机任务状态机设计：当前实现无 cancelled 枚举 25
5.8 司机位置上报时序设计：页面、Controller 与 Timer 26
5.9 乘客预约支付时序设计：当前支付逻辑直接位于 Controller 27
5.10 目标部署图：外部认证、实时通信、支付和推送尚未接入 28
5.11 前端组件设计：入口、路由、三端工作台与状态层 29
5.12 目标数据层组件图：当前为独立 Mock 仓库和内存状态 30
5.13 历史 StarUML 软件主界面截图 31
5.14 历史用例图设计过程截图 32
5.15 历史领域类图设计过程截图 33
5.16 历史枚举类图设计过程截图 34
5.17 历史状态图设计过程截图 35
5.18 历史时序图设计过程截图 36
5.19 历史部署图设计过程截图 37
5.20 历史组件图设计过程截图 38

8.1 历史 Java/JUnit 补充演示截图：不属于当前 Flutter 自动化测试 47
8.2 历史后台手测：新增车辆对话框 47
8.3 历史后台手测：新增车辆结果 48
8.4 历史后台手测：车辆编辑对话框 48
8.5 历史后台手测：车辆编辑结果 49
8.6 历史后台手测：司机新增与绑定结果 49
8.7 历史后台手测：智能建议生成、人工确认入口 49
8.8 历史后台手测：调度条件不足的提示 50
8.9 历史后台手测：自绘位置面板与固定客户 ETA 50
8.10 历史后台手测：位置变化局部截图 50
8.11 历史后台手测：运营分析界面之一 51
8.12 历史后台手测：运营分析界面之二 51

第一章　项目概述与实现范围

1.1　背景、目标和角色

跨校区、宿舍区、教学楼、科技园与附属医院之间存在持续的校园通勤需求。人工预约和电话派车容易造成余座信息不透明、通知滞后、调度缺乏数据依据和运行状态不可见。项目目标是以三类工作台将这些流程系统化，并提供课程所要求的计划、建模、实现、测试及配置管理证据。

角色	目标	当前功能
普通用户	查询校车、管理预约与费用	登录、浏览车次、选座、预约列表、模拟支付、取消、信用等级、本地通知
司机	明确任务并展示运行状态	登录、任务卡片、乘客名单与上车点、日周月统计、开始任务、模拟位置记录、到站完成
管理员	维护运力并辅助调度	车辆和司机维护、绑定校验、人工调度、规则式智能建议、确认生效、模拟位置监控、示例 ETA、历史统计

1.2　当前实现与生产能力的区别

能力	仓库中的实际实现	不应误认为已实现的内容
三端入口	编译期 `APP_MODE` 与页面入口切换	三套独立部署且同步数据的业务服务
认证	三个演示账号、450ms 模拟延迟与角色匹配	注册、真实身份验证、Token 和服务端授权
数据	各模块 Mock 仓库初始化独立内存状态	数据库持久化和跨浏览器、跨设备同步

支付	选择微信或支付宝标签，500ms 后更新本地状态	SDK 下单、扣款、支付回调、真实退款和对账
通知	`notificationProvider` 本地通知列表	FCM 或其他平台推送
定位监控	5 秒 Timer 增量模拟经纬度，自绘网格地图	GPS 权限与采集、后台定位、WebSocket 广播、地图 SDK
客户 ETA	固定的 1.2/2.6/4.8km、8/14/23 分钟示例	实际乘客定位、Haversine 计算和路网 ETA
智能调度	按需求人数降序，匹配空闲且已绑定司机的车辆	OpenAI 调用、全局最优、多车分拆或真实调度执行
Supabase	表结构、RLS 与种子 SQL 已保留	App 中可用的 Supabase 后端切换

管理员确认调度会修改后台自身状态，并增加后台位置记录，但不会创建乘客端车次或司机端任务。因而目前是三种业务流程的课程演示原型，不应称为跨客户端实时闭环系统。

第二章　项目计划与管理

课程工具标识： Microsoft Project / Project 类工具、CSV 导入表、甘特图、资源与成本计划

2.1　计划口径与时间差异

原计划说明记载 2026-04-16 至 2026-06-04、36 个工作日、400 工时、29 个任务和预算 551,700 元；CSV 及原最终报告采用 2026-04-16 至 2026-06-25 的课程交付周期。两份资料不能视为相同进度基线。

本报告用 CSV 展示三端原型的课程交付计划，同时保留原 Project 计划中的成本、资源与截图作为历史计划证据。6 月 4 日为原计划结束日期，6 月 25 日为课程提交计划日期；均不据此推断实际完成时间。CSV 中持续时间与开始、结束日期含周末，未经指定 Project 工作日历重算，不据此计算精确关键路径。

2.2　课程交付工作分解

ID	任务	工期	开始至结束（2026 年）	前置	责任资源
1	项目启动和选题确认	2d	04-16 至 04-17	—	项目负责人
2	项目范围和计划编制	5d	04-18 至 04-22	1	项目负责人
3	需求调研和角色分析	7d	04-23 至 04-29	2	需求分析人员
4	用例和业务规则建模	5d	04-30 至 05-04	3	需求分析人员
5	需求评审和需求基线	2d	05-05 至 05-06	4	负责人、需求人员
6	概要设计和架构选型	4d	05-07 至 05-10	5	开发人员
7	UML 和核心流程设计	5d	05-11 至 05-15	6	开发人员
8	数据模型和接口设计	4d	05-16 至 05-19	7	开发人员
9	普通用户端实现	7d	05-20 至 05-26	8	开发人员
10	司机端实现	5d	05-27 至 05-31	8	开发人员
11	后台车辆和司机维护	6d	06-01 至 06-06	8	开发人员
12	调度、监控、分析实现	6d	06-07 至 06-12	11	开发人员

13	支付信用通知及 UI 收口	4d	06-13 至 06-16	9	开发人员
14	自动化测试设计	2d	06-17 至 06-18	9, 10, 12, 13	测试人员
15	测试执行和缺陷修复	3d	06-19 至 06-21	14	测试、开发人员
16	配置管理和交付基线	1d	06-22	15	配置管理员
17	报告整理和证据汇总	3d	06-22 至 06-24	15	项目负责人
18	最终检查和提交	1d	06-25	16,17	项目负责人

可编辑证据位于 `report-assets/evidence/project_plan.csv` 和 `report-assets/evidence/project_plan.mpp`。CSV 是上述课程交付计划，MPP 是原 Project 计划；导入时需分别核对日历、任务依赖与资源，不能声称两者完全一致。

2.3 原 Project 计划与关键路径

阶段	原计划工期	原计划区间	工作内容
项目启动	8 工作日	04-16 至 04-27	需求分析、可行性研究
系统设计	6 工作日	04-28 至 05-05	概要设计、详细设计
编码实现	11 工作日	05-06 至 05-20	数据库、前端、后端逻辑、接口
测试	8 工作日	05-21 至 06-01	单元、集成、系统测试
部署上线	3 工作日	06-02 至 06-04	环境部署、项目验收
沟通管理	30.25 工作日	04-20 至 06-01	周会、评审、阶段汇报

原材料标注的阶段关键路径为“项目启动 → 系统设计 → 编码实现 → 测试 → 部署上线”，并列出可行性研究、详细设计、后端逻辑开发、接口开发、系统测试、环境部署为关键任务。这是原计划的描述，不是本次对 MPP 重新计算的结果。原里程碑为启动完成 4 月 27 日、设计完成 5 月 5 日、编码完成 5 月 20 日、测试完成 6 月 1 日、验收 6 月 3 日。



标识号	任务模式	任务名称	工期	开始时间	完成时间	前置任务	资源名称	成本
1		**项目启动**	**8 个工作日**	**2026年4月16日**	**2026年4月27日**			**¥52, 800. 00**
2		需求分析	3 个工作日	2026年4月16日	2026年4月23日		系统分析师, 办公	¥27, 600. 00
3		可行性研究	2 个工作日	2026年4月24日	2026年4月27日	2	系统分析师[50%	¥23, 200. 00
4		**系统设计**	**6 个工作日**	**2026年4月28日**	**2026年5月5日**	1		**¥80, 400. 00**
5		概要设计	2 个工作日	2026年4月28日	2026年4月29日		系统分析师, 办公	¥31, 400. 00
6		详细设计	4 个工作日	2026年4月30日	2026年5月5日	5	UI设计师, 系统分	¥48, 000. 00
7		**编码实现**	**11 个工作日**	**2026年5月6日**	**2026年5月20日**	4		**¥196, 800. 00**
8		数据库开发	5 个工作日	2026年5月6日	2026年5月12日		后端工程师, 开发	¥48, 000. 00
9		前端界面开发	6 个工作日	2026年5月6日	2026年5月13日		前端工程师, 开发服务器[1 台]	¥50, 400. 00
10		后端逻辑开发	8 个工作日	2026年5月6日	2026年5月15日		后端工程师, 开发服务器[1 台]	¥55, 200. 00
11		接口开发	3 个工作日	2026年5月18日	2026年5月20日	10, 8, 9	前端工程师[50%	¥43, 200. 00
12		**测试**	**8 个工作日**	**2026年5月21日**	**2026年6月1日**	7		**¥111, 000. 00**
13		单元测试	4 个工作日	2026年5月21日	2026年5月26日		测试工程师, 办公	¥39, 000. 00
14		集成测试	2 个工作日	2026年5月27日	2026年5月28日	13	测试工程师, 测试	¥35, 000. 00
15		系统测试	2 个工作日	2026年5月29日	2026年6月1日	14	测试工程师[50%	¥37, 000. 00
16		**部署上线**	**3 个工作日**	**2026年6月2日**	**2026年6月4日**	12		**¥110, 700. 00**
17		环境部署	3 个工作日	2026年6月2日	2026年6月4日		后端工程师, 开发	¥102, 700. 00
18		项目验收	2 个工作日	2026年6月2日	2026年6月3日		项目经理	¥8, 000. 00
19		**沟通管理**	**30. 25 个工作日**	**2026年4月20日**	**2026年6月1日**			**¥0. 00**
20		**项目周会**	**30. 25 个工作日**	**2026年4月20日**	**2026年6月1日**			**¥0. 00**
28		需求评审会						¥0. 00
29		阶段汇报						¥0. 00

图 2.1: 原 Project 工作分解与任务表：历史计划截图

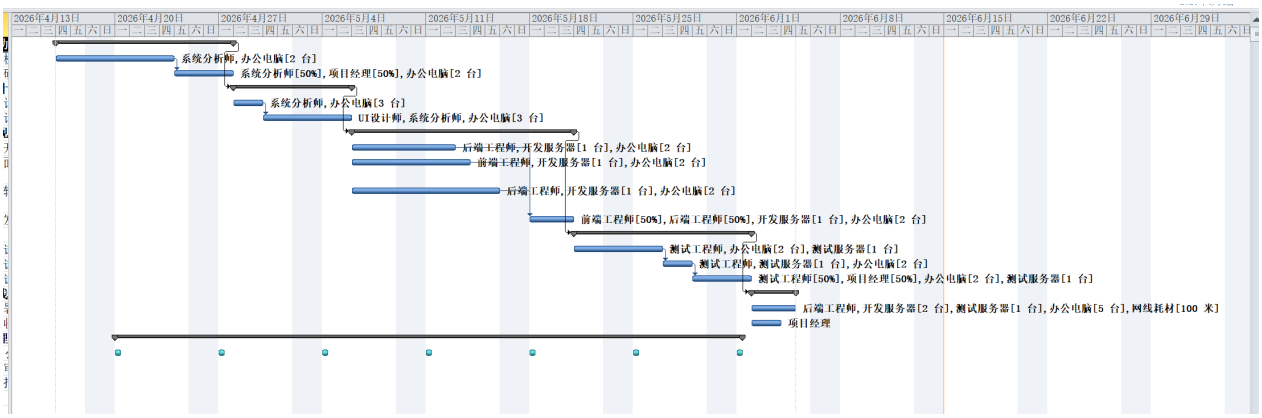


图 2.2: 原 Project 甘特图、依赖和资源分配：历史计划截图

2.4 资源和成本

人力资源	标准费率(元/工时)	加班费率(元/工时)	原计划最大单位

项目经理	500	750	100%
系统分析师	400	600	100%
前端工程师	300	450	100%
后端工程师	300	450	100%
测试工程师	250	375	100%
UI 设计师	350	525	100%

材料单价为开发服务器 20,000 元/台、测试服务器 15,000 元/台、办公电脑 8,000 元/台、网线耗材 5 元/米。它们是原计划模拟预算，并非真实采购或实际项目支出。

任务	资源分配	数量或比例
需求分析	系统分析师、办公电脑	100%、2 台
可行性研究	分析师、经理、电脑	各 50%、2 台
概要设计	分析师、电脑	100%、3 台
详细设计	UI 设计师、分析师、电脑	各 100%、3 台
数据库 / 前端 / 后端	对应工程师、开发服务器、电脑	工程师 100%、1 台、2 台
接口开发	前后端工程师、服务器、电脑	各 50%、1 台、2 台
单元 / 集成测试	测试工程师、测试服务器、电脑	100%、1 台、2 台
系统测试	测试工程师、经理、测试服务器、电脑	各 50%、1 台、2 台
环境部署	后端工程师、开发服务器、测试服务器、电脑、网线	100%、2 台、1 台、5 台、100 米
项目验收	项目经理	100%

原材料记录一个过度分配的工时资源，但未明确冲突人员与解决结果；后续重排计划需检查资源平衡。

阶段	预算（元）	子任务预算（元）
项目启动	52,800	需求分析 27,600；可行性研究 23,200
系统设计	80,400	概要设计 31,400；详细设计 48,000
编码实现	196,800	数据库 48,000；前端 50,400；后端 55,200；接口 43,200

测试	111,000	单元 39,000；集成 35,000；系统 37,000
部署上线	110,700	环境部署 102,700；验收 8,000
总计	551,700	原计划总预算，不含真实结算信息

需求分析另记固定成本 2,000 元，概要设计另记 1,000 元；其他列出的任务固定成本为 0。阶段汇总与子任务金额存在差额，原表未完整说明计算口径，不能简单相加后称为已审计预算。编码阶段占原总预算约 35.7%。现金流峰值记录为 6 月 1 日当周 129,200 元；完整历史现金流图如下。

现金流量 3 2026年6月15日
项目2

	2026年4月13日	2026年4月20日	2026年4月27日	2026年5月4日	2026年5月11日	2026年5月18日	2026年5月25日	2026年6月1日	2026年6月
项目启动	¥500.00	¥1,250.00	¥250.00						
需求分析	¥12,400.00	¥15,200.00							
可行性研究		¥11,600.00	¥11,600.00						
系统设计			¥666.67	¥333.33					
概要设计			¥31,400.00						
详细设计			¥24,000.00	¥24,000.00					
编码实现									
数据库开发				¥28,800.00	¥19,200.00				
前端界面开发				¥25,200.00	¥25,200.00				
后端逻辑开发				¥20,700.00	¥34,500.00				
接口开发						¥43,200.00			
测试									
单元测试						¥19,500.00	¥19,500.00		
集成测试							¥35,000.00		
系统测试							¥18,500.00	¥18,500.00	
部署上线									
环境部署								¥102,700.00	
项目验收								¥8,000.00	
沟通管理									
项目周会									
总和	¥12,900.00	¥28,050.00	¥67,916.67	¥99,033.33	¥78,900.00	¥62,700.00	¥73,000.00	¥129,200.00	

图 2.3: 原计划按周现金流：保留原预算分布，不作为真实财务记录

2.5 沟通、质量和风险管理

每周项目组会议形成纪要；阶段节点开展需求评审并形成评审报告；里程碑向项目负责人汇报并形成阶段报告。原计划的交叉评审方式可保留，但其中“Java 开发手册”不适用于该 Flutter 工程；当前规范以 `analysis_options.yaml` 引入的 `flutter_lints` 为准。原覆盖率不低于 80% 是计划目标，没有现成覆盖率证据证明达标。

风险	描述	原概率 / 影响	应对措施
技术	新技术学习成本	中 / 高	提前培训、先做原型、保留缓冲时间
人员	核心人员离开	低 / 高	文档化、版本管理和知识共享
进度	需求频繁变更	高 / 中	评估范围、记录需求变更、同步测试

验证	模拟数据被误称为生产能力	已知限制	区分原型、历史证据与未来服务；不以截图替代实测

第三章　需求规格与验收

课程工具标识：StarUML、需求规格说明、需求追踪矩阵

3.1　业务需求和用例

业务目标包括提升预约效率（BRQ-01）、提高任务透明度（BRQ-02）、支持调度决策（BRQ-03）、掌握运行状态（BRQ-04）、提供运营复盘（BRQ-05）、满足课程交付（BRQ-06）。其中运行状态和运营分析当前采用演示数据。

编号	用例	参与者	触发与输出
UC-01	登录系统	全部角色	账号密码与入口匹配后进入工作台
UC-02	查看车次	普通用户	展示路线、时间、票价、余座
UC-03	预约车次	普通用户	选择可用座位、创建预约
UC-04	支付车费	普通用户	待支付订单选择方式、模拟完成
UC-05	取消预约	普通用户	取消待发车预约、记录信用扣分
UC-06	查看信用等级	普通用户	展示信用分、等级和取消记录
UC-07	查看通知	普通用户	查看预约、支付、信用通知
UC-08	查看派车任务	司机	查看今日任务卡片
UC-09	查看乘客名单	司机	展示姓名、电话尾号、上车点、座位
UC-10	日周月统计	司机	切换任务、人次、里程、准点率
UC-11	共享位置	司机	开始任务后追加模拟位置记录
UC-12	车辆管理	管理员	新增、编辑、删除车辆
UC-13	司机管理	管理员	维护司机档案与车辆绑定
UC-14	人工调度	管理员	按需求选择车辆、司机并生成方案
UC-15	智能辅助调度	管理员	本地规则生成推荐，人工确认
UC-16	位置监控	管理员	选择模拟标记查看车辆、司机、速度
UC-17	查看客户 ETA	管理员	展示固定客户距离和时间示例

UC-18	历史数据分析	管理员	切换周期查看人次、收入、利用率、热度

原 UML 使用 include/extend 表达预约与查询、支付与预约、通知与取消、智能建议与人工调度的关系。它们描述设计意图，不代表后台与司机端已经连接实时服务。

3.2 功能需求

编号	功能	输入及处理	输出 / 当前范围
FR-01	登录与角色入口	手机、密码、入口模式，校验账号和角色	工作台或错误提示；无注册
FR-02	预约选座	用户、车次、座位，检查时刻、余座、重复	待发车且待支付的预约
FR-03	预约查看筛选	用户 ID、预约状态	对应订单列表
FR-04	取消与信用扣分	预约 ID，检查状态与发车时间	已取消、扣分记录、通知
FR-05	车费支付	预约 ID、模拟支付方式	支付记录与本地支付通知
FR-06	信用与通知	信用分、取消记录、通知状态	信用面板与本地通知中心
FR-07	任务及乘客名单	仓库中的司机任务	路线、车次、上车点和座位
FR-08	司机统计	日、周、月	完成车次、人次、里程、准点率
FR-09	位置共享	开始 / 完成任务	每 5 秒模拟位置记录，结束后停止
FR-10	车辆管理	车牌、车型、座位数、状态	更新列表及校验错误
FR-11	司机管理	姓名、手机、驾照、车辆绑定	档案列表与一对一绑定校验
FR-12	人工与智能调度	待调度需求、空闲车辆、司机	待确认建议及确认后的后台状态
FR-13	位置与 ETA	后台模拟位置、固定客户样例	自绘地图标记和示例 ETA
FR-14	运营分析	日、周、月统计周期	演示趋势、人次、收入、利用率、热度

FR-06、FR-08、FR-14 为原文中等优先级，其余为高优先级。真实地图、乘客位置上传、CSV 数据导出、自定义统计时间段属于扩展需求，当前并未全部实现。

3.3 业务规则与状态语义

规则	当前约束	关联模块
RUL-01	账号角色必须与当前入口一致	MockAuthRepository、路由
RUL-02	只有 scheduled 且未过发车时刻的车次可预约	TripModel.isBookable
RUL-03	同用户同车次不能存在重复 active 预约；座位必须可选	`PassengerBookingController`
RUL-04	只有 pending 且未发车的预约可取消	cancelBooking
RUL-05	距发车整数分钟数不大于 120 扣 5 分，否则扣 2 分；信用分限制 0–100	取消规则、AuthController
RUL-06	UI 仅对 pending 且 unpaid 的订单开放支付	BookingModel.canPay
RUL-07	运行中车辆不可删除	`AdminController.deleteVehicle`
RUL-08	有后台位置记录的司机不可删除	`AdminController.deleteDriver`
RUL-09	车辆与司机保持一对一绑定；司机手机不能重复	驾驶员维护校验
RUL-10	人工调度要求空闲车辆、足够座位、合法司机绑定	`createManualDispatchPlan`
RUL-11	智能调度仅选空闲且已绑定司机车辆，同批不重复用车	`generateAiDispatchPlans`
RUL-12	一名司机的本地任务状态只能有一个 activeTaskId	DriverTaskController

支付入口的 `canPay` 比 Controller 中的部分检查更严格；Controller 不是服务端授权边界，正式后端仍需独立做幂等与并发校验。已支付预约在当前代码中仍是 `BookingStatus.pending`，支付结果由独立的 `BookingPaymentStatus.paid` 表示；不能使用原方案中的 `BookingStatus.paid` 代替。

3.4 非功能需求与验证方式

编号	类别	要求	证据或限制
NFR-01	易用性	卡片式工作台、明确的核心按钮	Widget 测试、历史界面截图
NFR-02	可维护性	core/features/shared 分层，业务放 Notifier	源码结构核对
NFR-03	可测试性	核心主流程和负向业务规则可自动验证	Flutter Test
NFR-04	可扩展性	数据初始化与页面分离	Mock 仓库；真实仓库尚未接入
NFR-05	响应式	LayoutBuilder 适配布局	当前页面有实现，仍需多设备验证
NFR-06	时序	5 秒模拟定位、500ms 模拟支付	代码常量，不是性能保证
NFR-07	安全	正式版需加密、权限、审计、隐私保护	原型不提供生产安全保证

3.5 需求追踪与验收标准

需求	主要实现位置	测试对应
FR-01	auth、core/providers/auth、core/router	三种登录 Widget；role entrance mismatch
FR-02–03	passenger/booking、passenger_home_page	选座生成订单；duplicate booking
FR-04	cancelBooking、adjustCreditScore	信用 96→91；completed/cancelled 拒绝取消
FR-05–06	payBooking、notificationProvider	支付成功与本地通知；不代表真实扣款
FR-07–09	driver/tasks、driver_home_page	司机专项 57 项与主流程 Widget
FR-10–11	adminProvider、车辆与司机模型	编辑、新增；运行车辆删除、绑定冲突

FR-12	dispatch 模型、AdminController	生成确认；座位不足、无可用车辆
FR-13	admin_home_page、location 模型	标记点击、模拟刷新、固定 ETA 展示
FR-14	analytics 模型与后台统计面板	周统计切换与路线热度展示

验收流程分别为乘客“登录 → 浏览 → 预约 → 支付或取消”、司机“登录 → 查看任务 → 开始 → 模拟定位 → 完成”、管理员“维护运力 → 生成建议 → 确认 → 查看监控”。UML 验收按实际保留的 12 张图计算，而非原需求验收表误写的 10 张。文档验收改为本综合报告完整覆盖七类课程内容，不再要求七份重复文件。测试通过仅说明被覆盖场景，不能等同全部需求已经被充分验证。

第四章　总体架构与详细设计

课程工具标识：Flutter / Dart、Riverpod、GoRouter、StarUML

4.1　真实依赖与运行入口

项目	声明或锁定版本	用途
Dart	SDK 约束 `^3.11.5`	语言与工具链要求
Flutter	工程使用 Flutter SDK	六平台工程；不代表六平台均经测试
flutter_riverpod	声明及锁定 3.3.1	Provider、Notifier 与状态订阅
go_router	声明及锁定 17.2.2	登录跳转与角色路由
cupertino_icons	声明 1.0.8，原锁定 1.0.9	图标
flutter_lints	6.0.0	分析规则
flutter_test	Flutter SDK 内置	Widget、业务状态与模型测试

当前未引入 Dio、Retrofit、Hive、secure storage、socket_io_client、geolocator、flutter_map、fl_chart、pay、firebase_messaging、supabase_flutter、freezed 或 build_runner；这些仅是初期选型候选。

```
// lib/main.dart: actual initialization pattern
const rawMode = String.fromEnvironment(
  'APP_MODE', defaultValue: 'passenger');
final initialMode = AppMode.fromValue(rawMode);
runApp(ProviderScope(
  overrides: [initialAppModeProvider.overrideWithValue(initialMode)],
  child: const AppRoot(),
));
```

`lib/app.dart` 观察入口和路由 Provider，构建 `MaterialApp.router`。`lib/core/router/app_router.dart` 观察入口模式和认证状态：未登录或角色不匹配返回登录页，已登录且路径不属于本角色时重定向到对应工作台。`APP_MODE` 是 dart-define 编译参数，不是运行时读取的 shell 环境变量。

4.2 模块边界和数据流

边界	关键实现	职责
入口与路由	main.dart、app.dart、appRouterProvider	初始化、主题、登录态和角色导航
认证	AuthRepository 接口、MockAuthRepository、AuthController	演示账号校验、登录退出、信用更新
乘客	MockPassengerRepository、PassengerBookingController	车次、预约、支付、取消规则
司机	MockDriverRepository、DriverTaskController	任务、统计、位置 Timer 生命周期
后台	MockAdminRepository、AdminController	车辆司机、需求、建议、监控、分析
共享	core/models、notificationProvider、shared/widgets	值对象、通知、入口切换、公共卡片

数据流是“页面触发操作 → Notifier 校验与生成新模型 → 更新 state → 页面重建”。仓库提供初始演示数据，后续操作主要更新 Provider 内存状态，不是把所有写操作提交给数据仓库或数据库。认证定义了可替换接口，其他三个模块当前直接依赖对应 Mock 类型；不存在已经完成的统一 `BackendBootstrap` 服务。

4.3 核心数据字典

以下模型位于 `lib/core/models/`；字段为核心摘要，以具体 Dart 定义为最终依据。

模型	职责及主要字段
UserModel	用户身份与信用：id、name、phone、creditScore、role

TripModel	路线与座位：id、routeName、origin、destination、vehicleId、driverId、departureTime、totalSeats、bookedSeats、fare、status；isBookable 为计算属性
BookingModel	预约：id、userId、tripId、seatNo、status、fare、createdAt、cancelledAt、creditPenalty、paymentStatus、paymentMethod
PaymentModel	支付记录：id、bookingId、amount、method、status、createdAt、completedAt
DriverTaskModel	任务：id、tripNo、routeName、起终点、vehicleId、plateNo、departureTime、distanceKm、status、passengers
PassengerPickup	上车信息：name、phoneSuffix、pickupPoint、seatNo
DriverStats	统计：period、totalTrips、totalPassengers、totalDistance、onTimeRate
LocationUpdate	司机模拟位置：vehicleId、lat、lng、speed、timestamp
VehicleModel	车辆：id、plateNo、model、seatCount、status
AdminDriverModel	档案：id、name、phone、licenseNo、boundVehicleId
DispatchDemandModel	需求：id、routeName、origin、destination、passengerCount、departureTime、status
DispatchPlanModel	建议：id、demandId、routeName、vehicleId、driverId、passengerCount、departureTime、loadRate、isAiGenerated、isConfirmed
VehicleLocationModel	后台位置：vehicleId、tripNo、driverId、lat、lng、speed、updatedAt
AdminAnalyticsSnapshot	运营统计：period、totalPassengers、totalRevenue、vehicleUtilization、趋势和路线排名
AppNotificationModel	本地通知：标识、标题、正文、类型、时间、已读状态

用户和车次分别与多个预约关联；预约与支付记录通过 bookingId 关联；司机任务包含乘客上车信息及位置记录；司机可绑定一辆车；需求与建议、建议与车辆和司机通过 ID 关联。模型关系不意味着这些模块的演示数据会自动同步。

4.4 枚举与状态转换

当前类型	字面值
UserRole / AppMode	passenger、driver、admin
TripStatus	scheduled、running、completed、cancelled
BookingStatus	pending、completed、cancelled
BookingPaymentStatus	unpaid、processing、paid、refunded
PaymentMethod	wechat、alipay

PaymentStatus	processing、paid、failed、refunded
DriverTaskStatus	pending、running、completed
VehicleStatus	idle、running、maintenance
DispatchDemandStatus	pending、assigned
AnalyticsPeriod	day、week、month

原 UML 中部分枚举来自早期方案，例如预约 paid、任务 cancelled、分析 daily/weekly/monthly。保留图像用于展示原建模过程，但解释当前状态必须按上表和源码。

预约创建后为 pending + unpaid；模拟支付经 processing 到 paid，但预约仍为 pending。取消将预约改为 cancelled，若已支付则把预约支付状态标记 refunded；没有真实退款网关，且不能据此声称支付流水已经真实对账。司机任务 pending → running → completed，通过 activeTaskId 限制同时进行的任务。车次模型支持生命周期状态，但目前并没有由司机任务自动推动乘客车次状态的跨模块链路。

4.5 乘客流程

1. 浏览车次：判断发车时刻和状态。剩余座位为总座位减仓库预占 bookedSeats，再减新 active 预约数；可选座位从 bookedSeats+1 起排除已预约座位。
2. 创建预约：依次检查可预约、未满座、无同用户重复预约、座位仍可用；新记录插入列表并产生本地预约通知。
3. 支付：生成 processing 流水并标记订单支付中，延迟 500ms 后重新检查订单未取消，再标记支付完成和推送本地通知。
4. 取消：检查预约存在、pending、未发车；计算取消扣分，更新状态、当前登录用户信用及本地通知。信用分限制在 0–100。

正式实现还需处理双击支付、重复回调、并发占座、支付中取消、退款和订单归属校验。目前 Controller 方法不是生产 API，不提供完整的服务端并发和安全约束。

4.6 司机任务与定位流程

`startTask` 检查 activeTaskId 和目标任务，标记 running 后立即追加第一条位置，并启动 5 秒周期 Timer。位置从纬度 31.2304、经度 121.4737 开始按记录序号递增，速度也由固定规则生成。`simulateNextLocationReport` 提供手动模拟按钮；新位置前插，`latestLocation` 返回第一项。`completeActiveTask` 标记 completed、取消 Timer 并清除 activeTaskId。`ref.onDispose` 同样取消 Timer。司机统计是仓库提供的日周月样例，不是对所有实时任务持续聚合后的数据。

4.7 后台管理、智能调度和分析

车辆新增检查车牌、车型、座位数及重复车牌；编辑检查目标存在及车牌冲突；删除运行车辆被拦截，删除其他车辆时清理司机绑定和后台位置。司机新增检查完整性、重复手机及车辆一对一绑定，后台位置列表中出现的司机不允许删除。

人工调度校验待处理需求、空闲车辆、司机及容量。智能建议按人数降序处理需求，对每项选择当前候选列表中第一辆未使用且容量足够的车，候选必须为空闲且已有绑定司机；无法覆盖的需求被跳过。这个算法并不拆分单个大需求到多辆车，也不搜索全局最优解。生成建议仅创建待确认模型。

确认方案再次检查记录完整和车辆仍空闲，然后标记方案已确认、需求 assigned、车辆 running，必要时绑定司机，并创建后台模拟位置。位置 Timer 每 5 秒仅更新运行车辆的坐标。地图使用 `CustomPainter` 绘制网格，统计图表也使用 Flutter 自身组件，并未依赖真实地图或 fl_chart。

第五章　UML 建模与图文证据

课程工具标识：StarUML 原生模型、PNG 导出、设计过程截图

5.1　模型文件和适用范围

可编辑 StarUML 原文件在 `report-assets/evidence/BUS-FLUTTER-UML.mdj`。实际保留 12 张 UML 图：用例 2、类图 2、状态图 3、时序图 2、部署图 1、组件图 2。历史资料称使用 StarUML 6.3.0；截图可见试用版界面，本文不再沿用原文“开源软件”的错误表述。图中的外部支付、推送、数据库和 WebSocket 为目标部署示意；`PaymentService`、`BackendBootstrap` 等名称也是设计抽象，不应据此寻找当前仓库中不存在的运行服务。

5.2 用例与领域模型

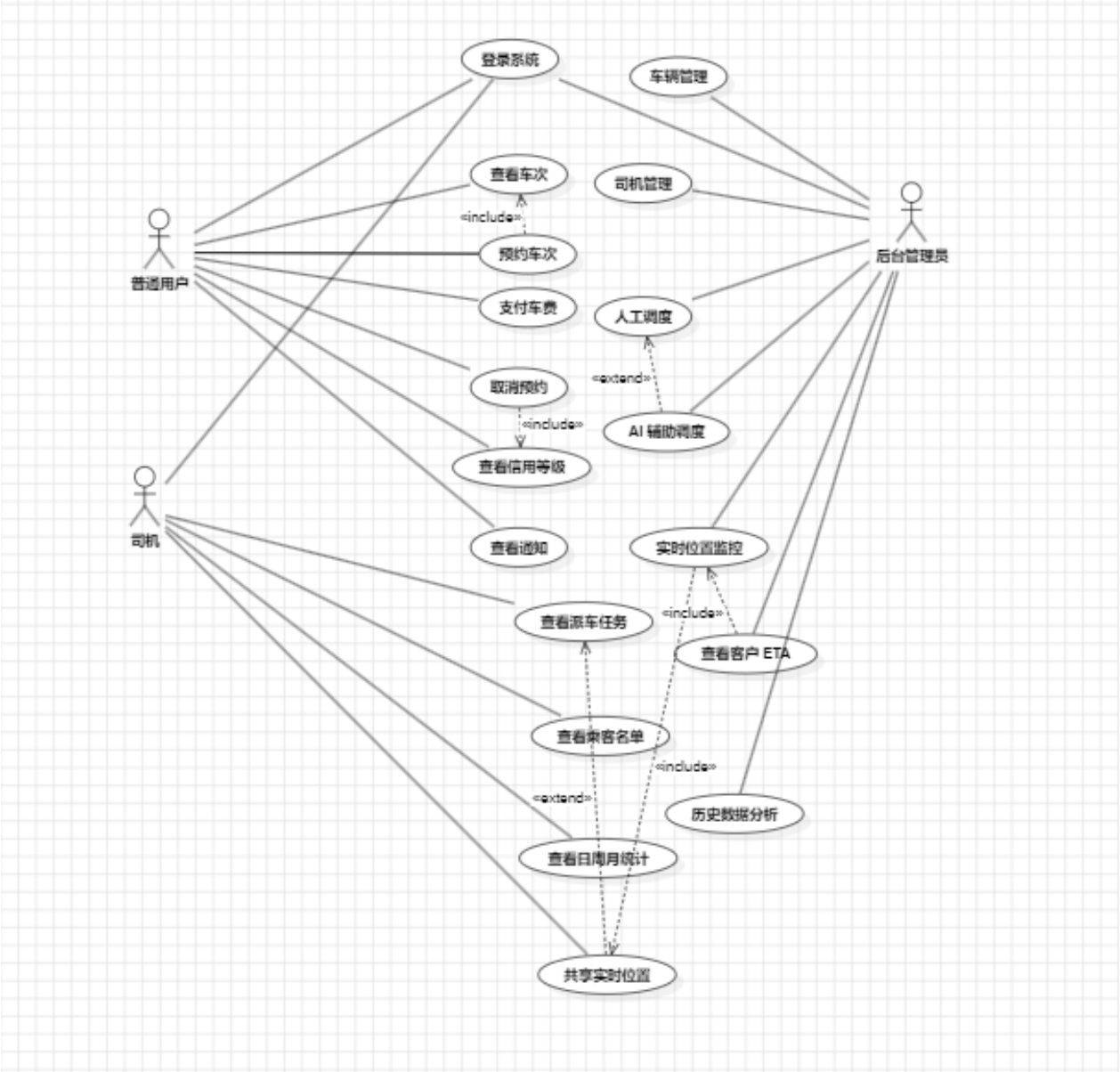


图 5.1: 系统总览用例图：普通用户、司机与管理员

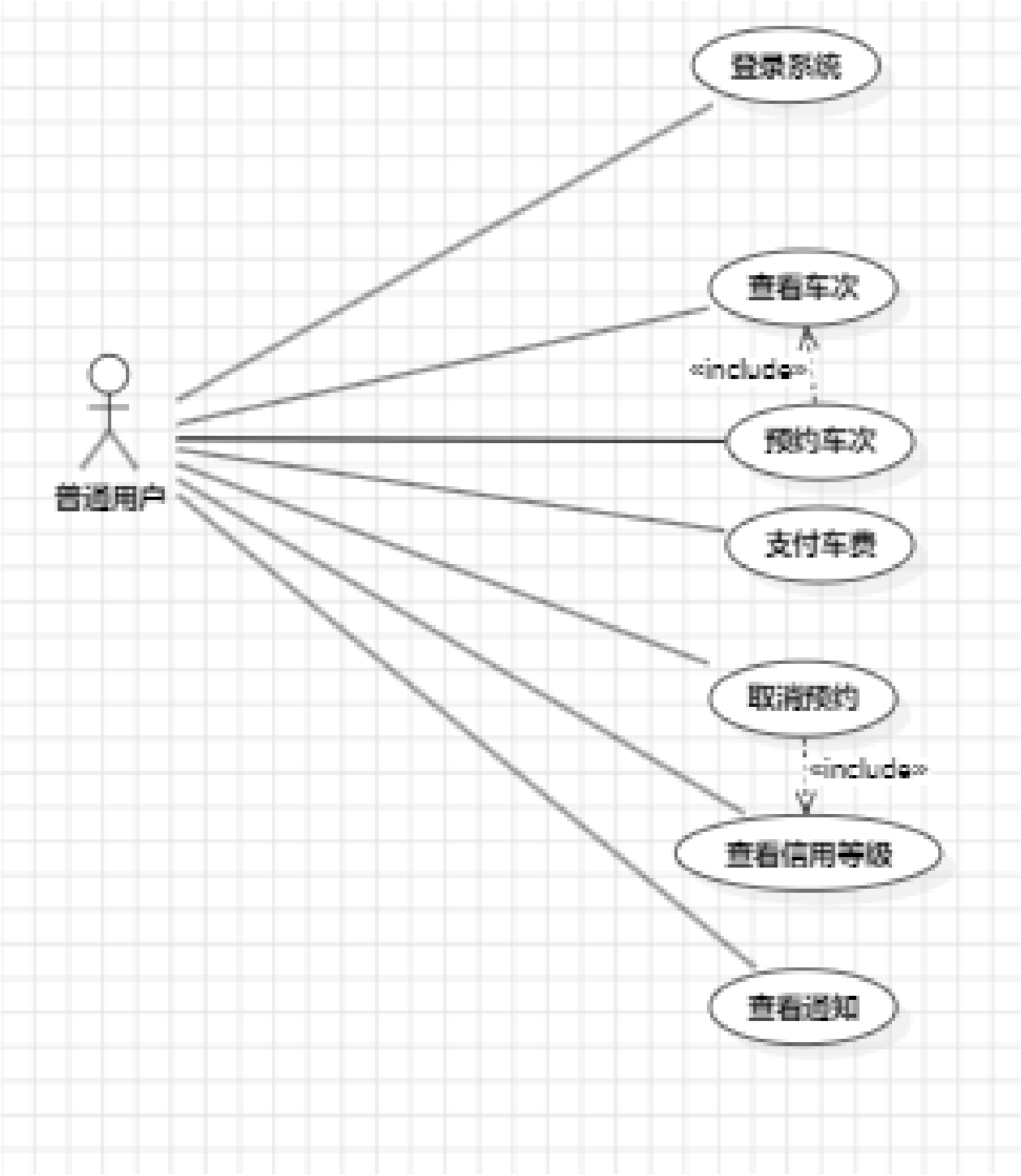


图 5.2: 乘客端用例详图：查询、选座、支付、取消与通知

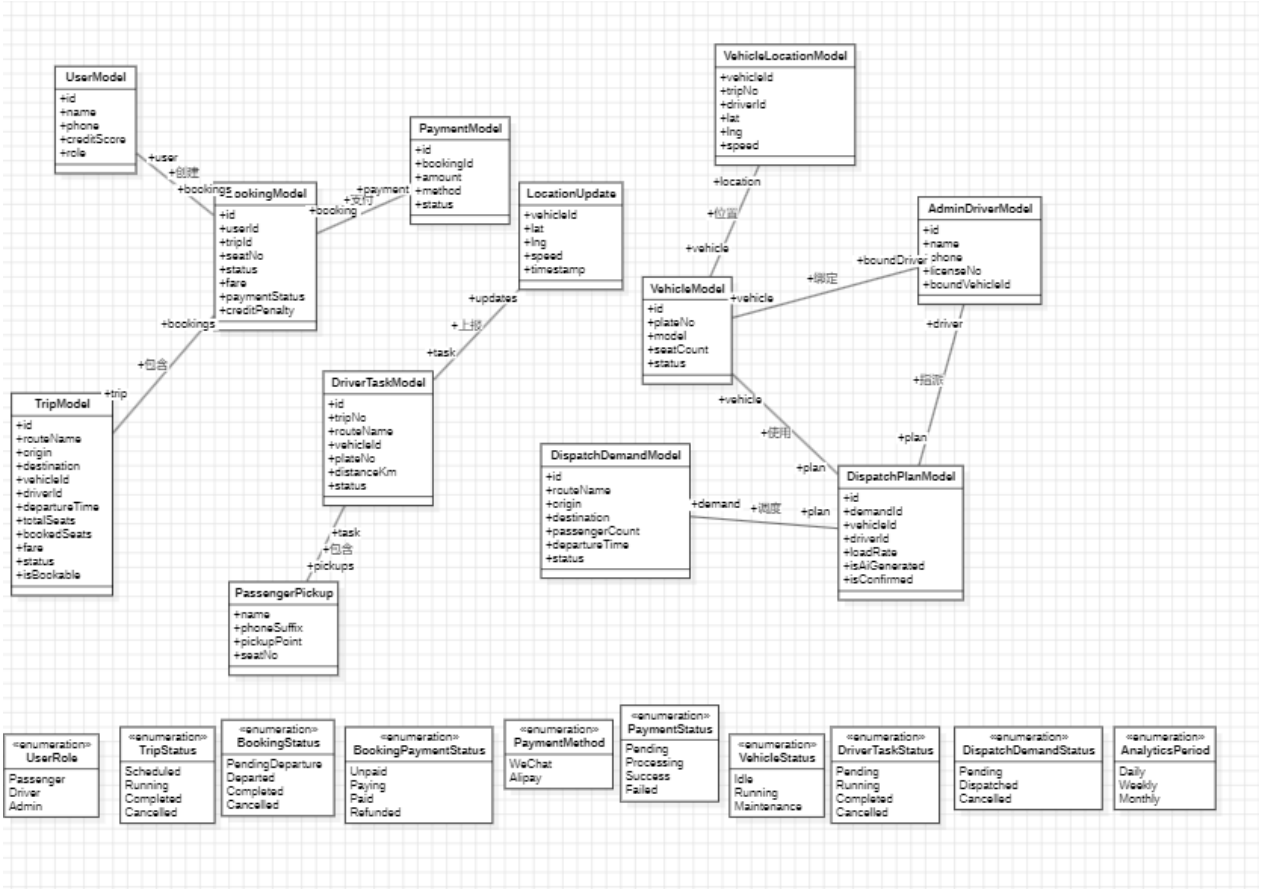


图 5.3: 历史核心领域类图：当前字段和状态以第四章修订说明为准

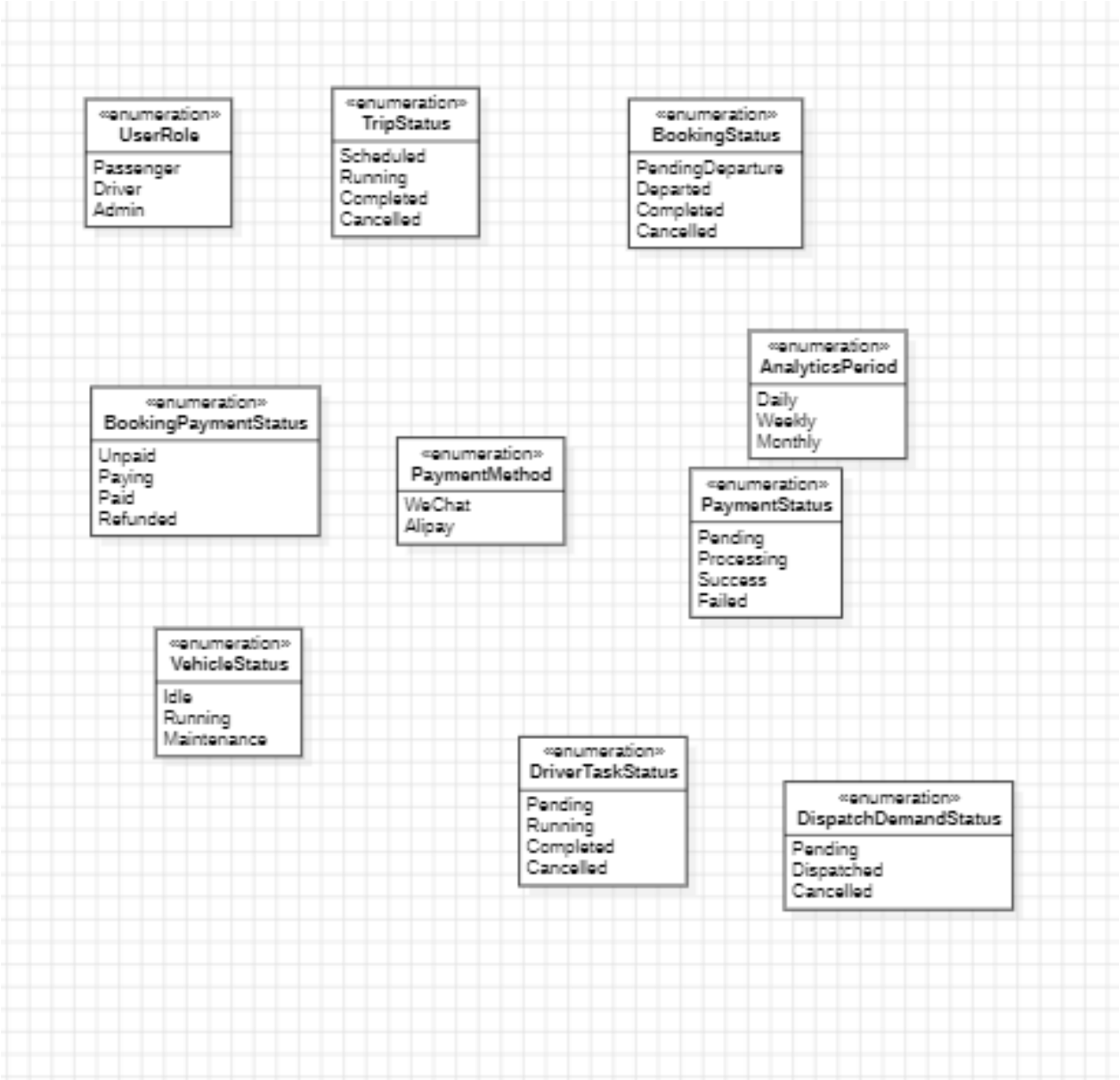


图 5.4: 历史枚举与业务类型图：包含早期方案的状态命名

5.3 生命周期、交互与部署

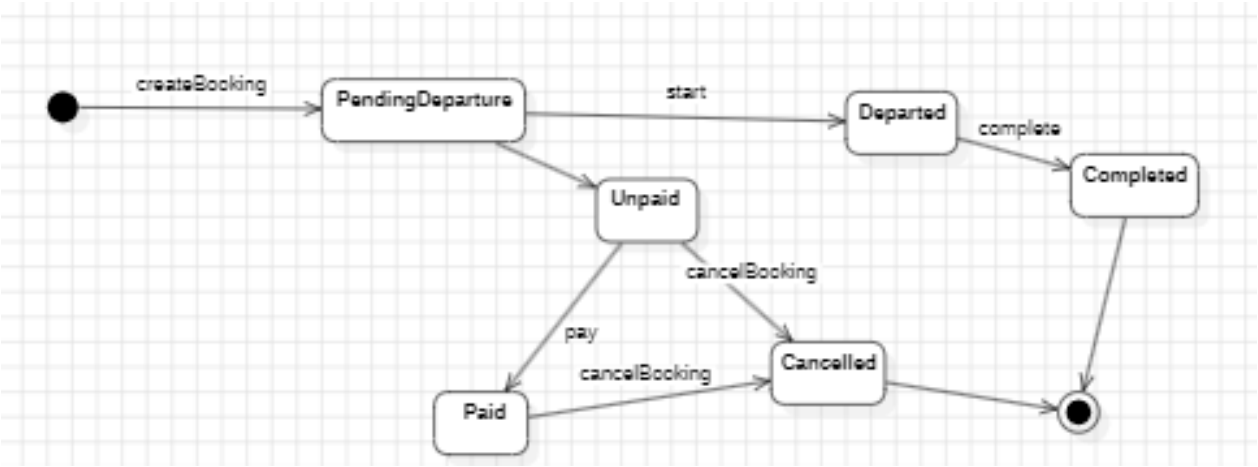


图 5.5: 预约状态机设计意图：当前预约状态与支付状态分开存储

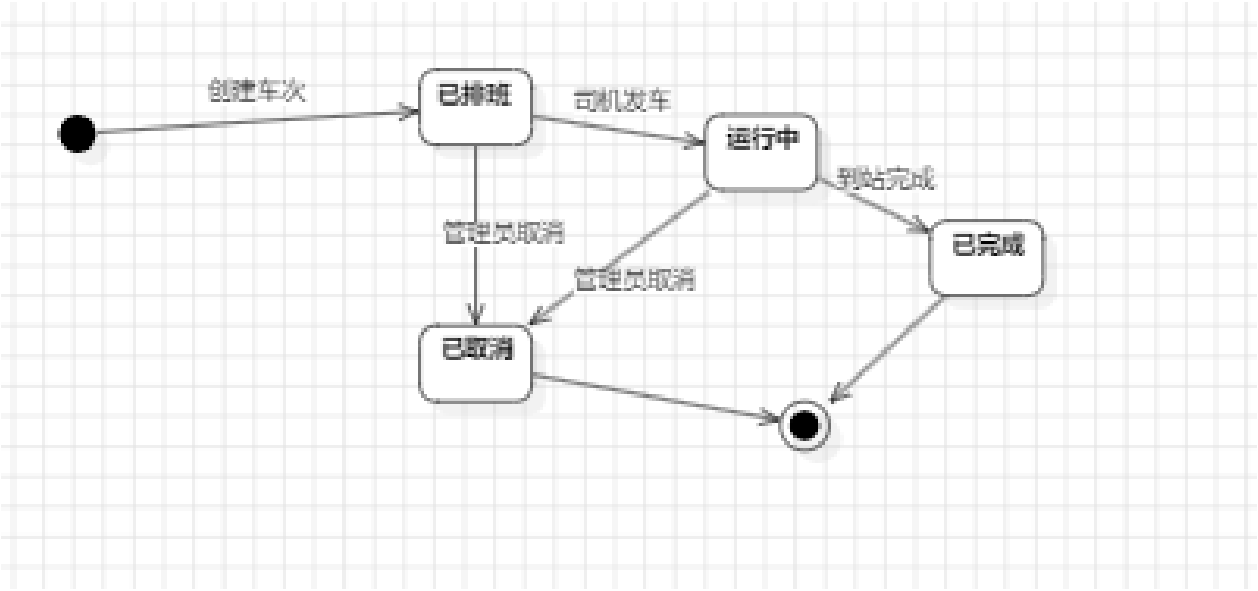


图 5.6: 车次状态机设计：scheduled、running、completed、cancelled

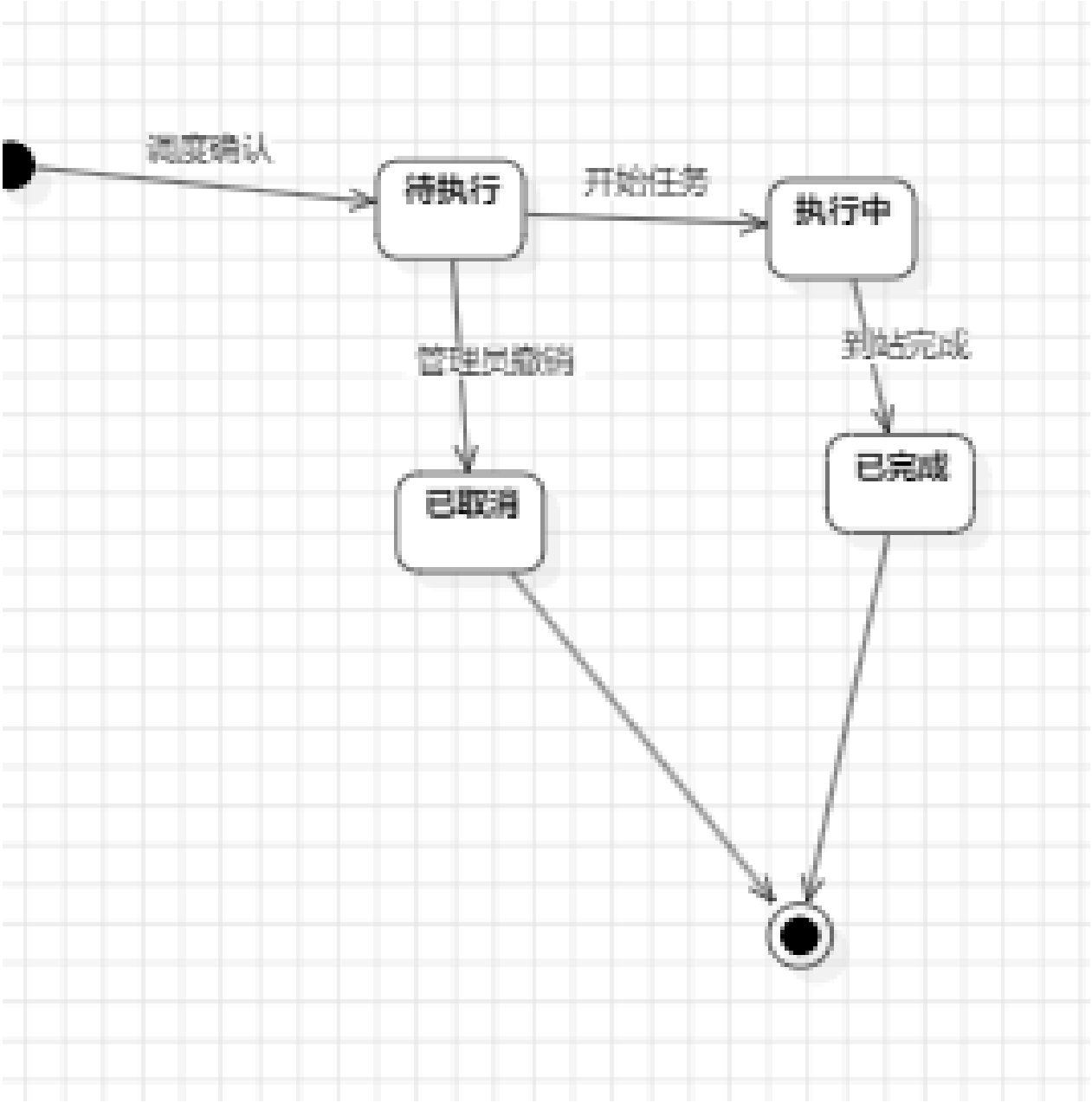


图 5.7: 司机任务状态机设计：当前实现无 cancelled 枚举

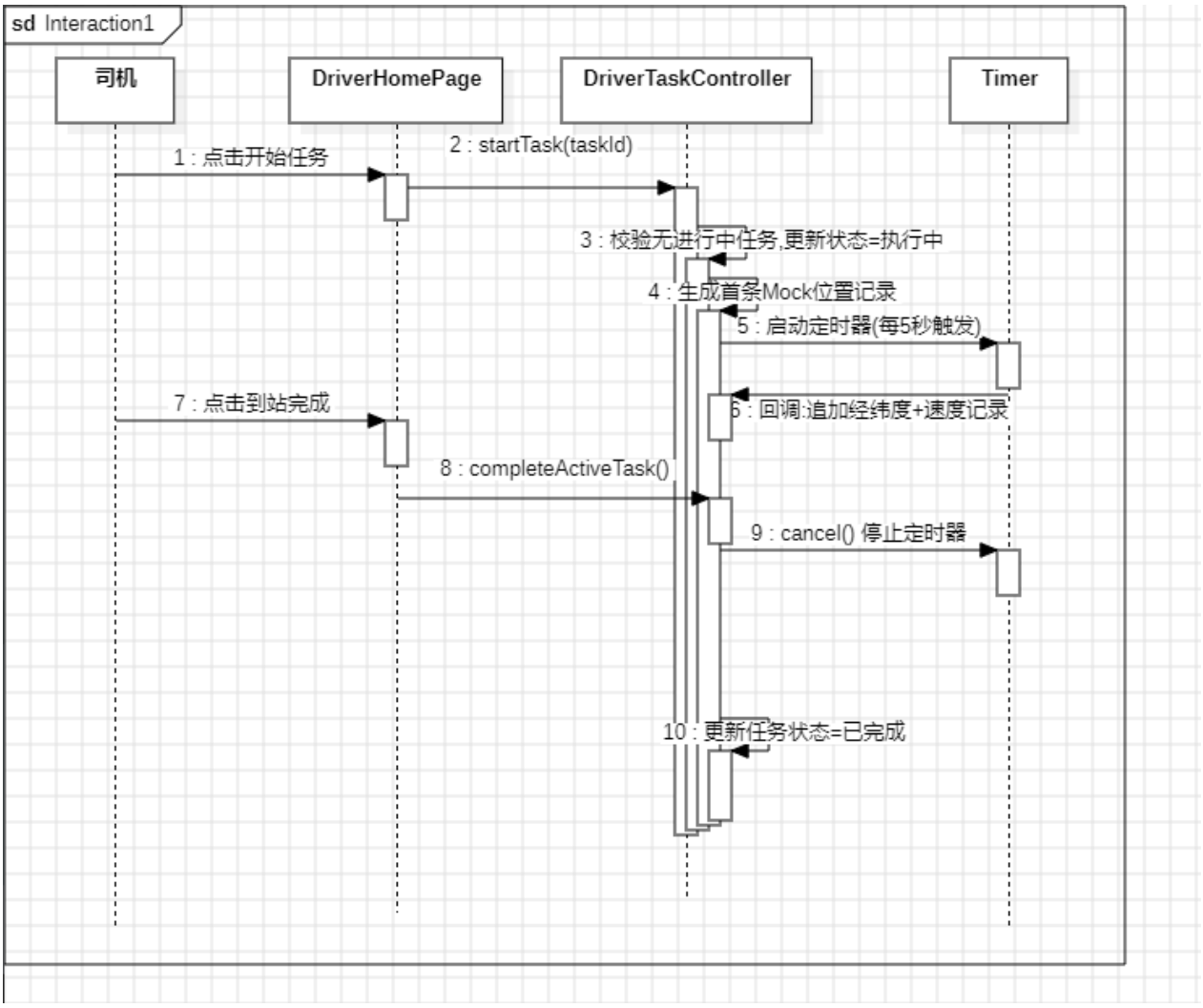


图 5.8: 司机位置上报时序设计：页面、Controller 与 Timer

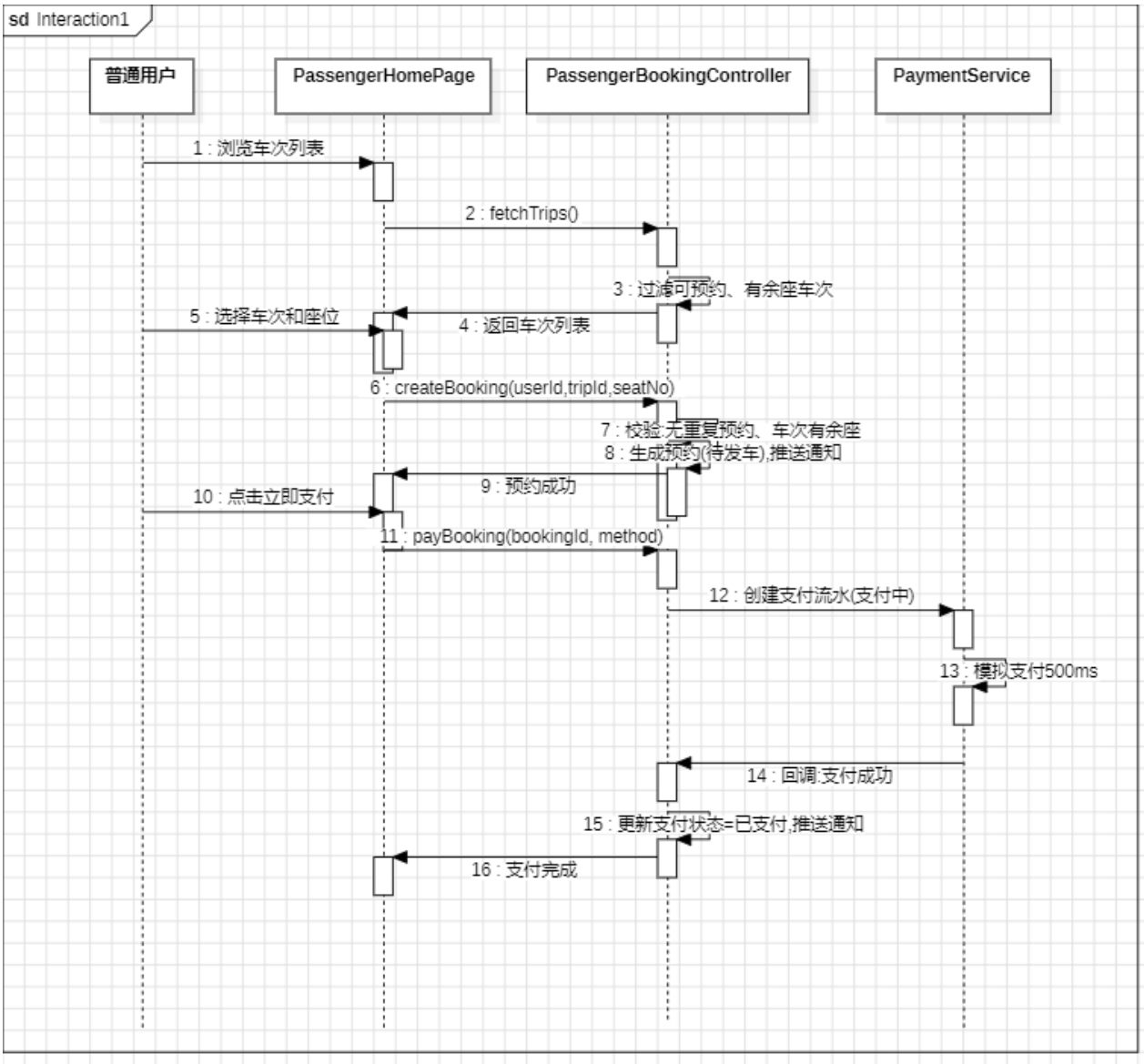


图 5.9: 乘客预约支付时序设计：当前支付逻辑直接位于 Controller

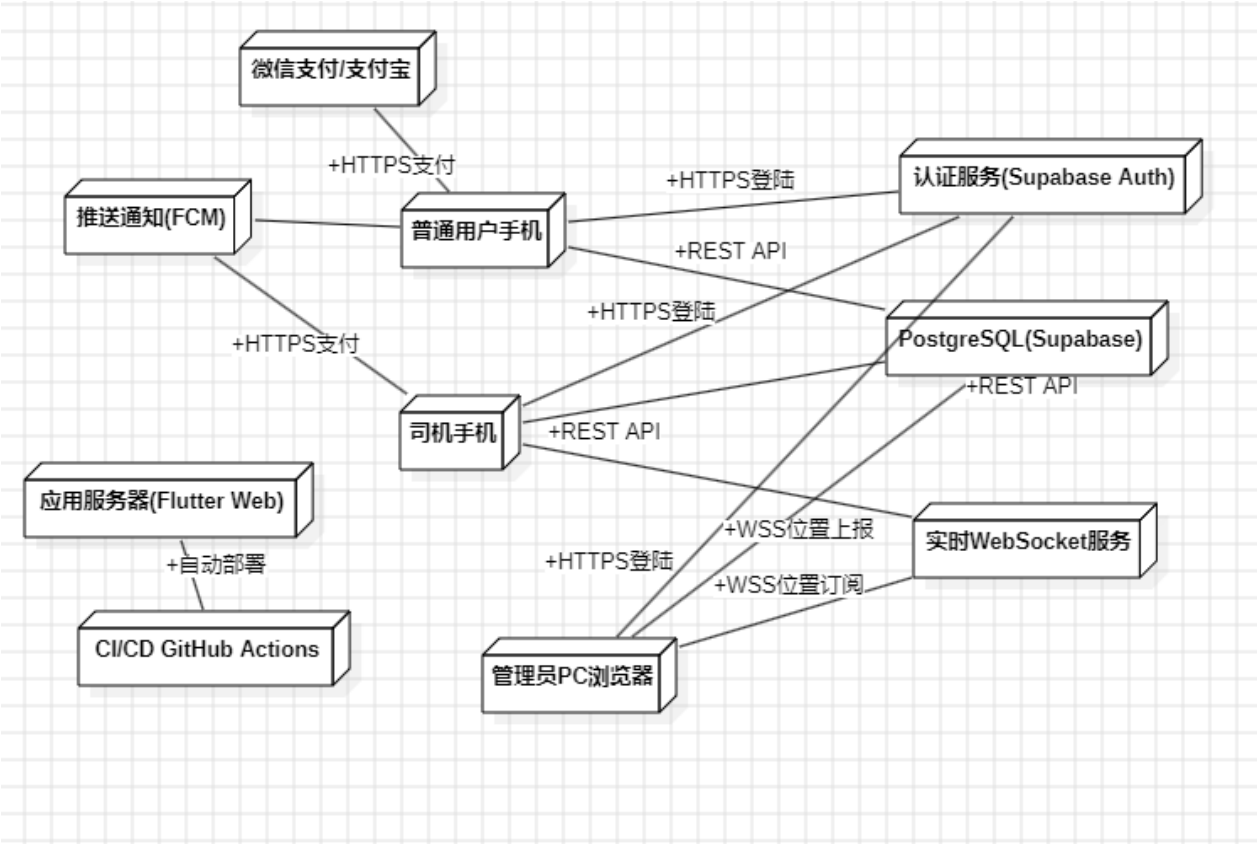


图 5.10: 目标部署图：外部认证、实时通信、支付和推送尚未接入

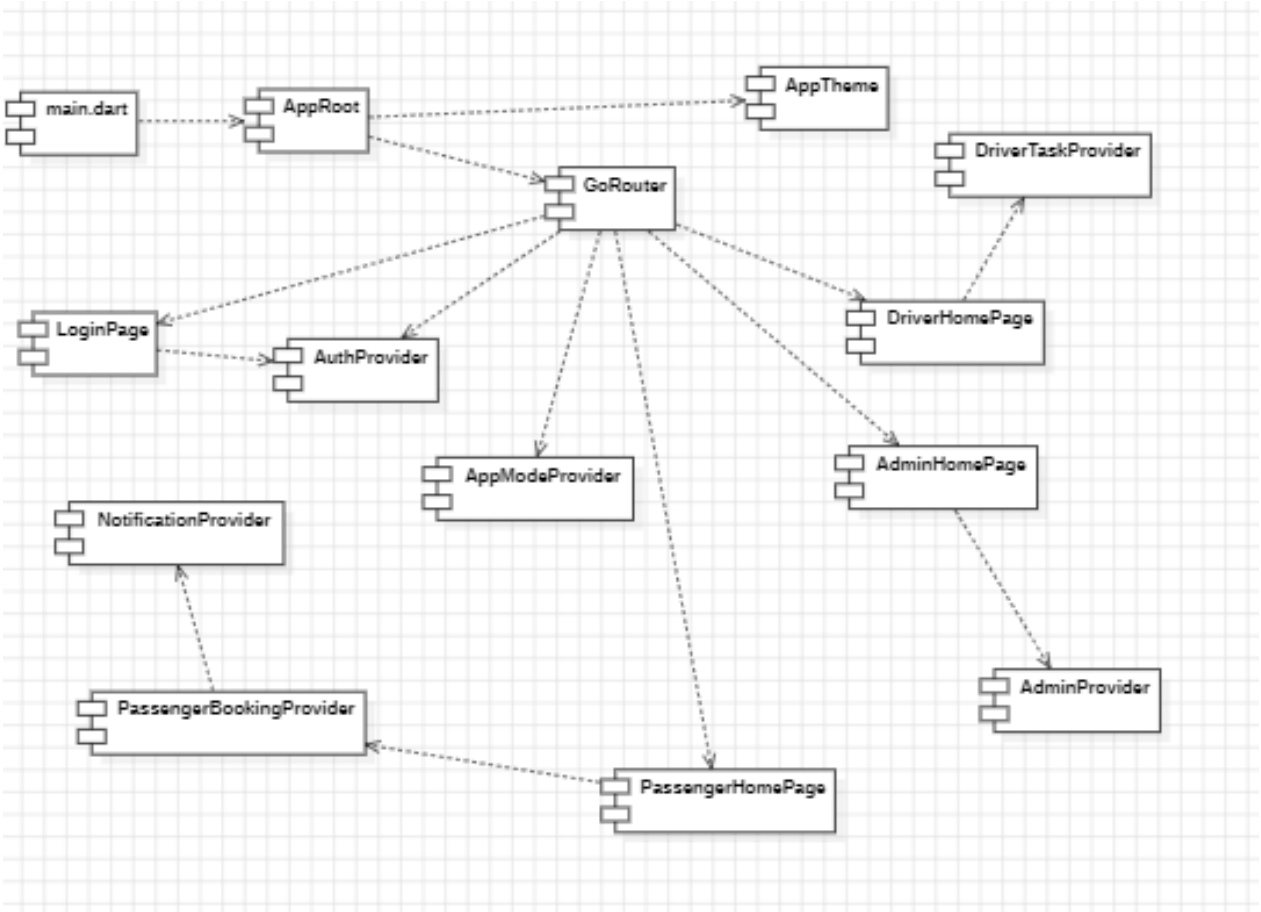


图 5.11: 前端组件设计：入口、路由、三端工作台与状态层

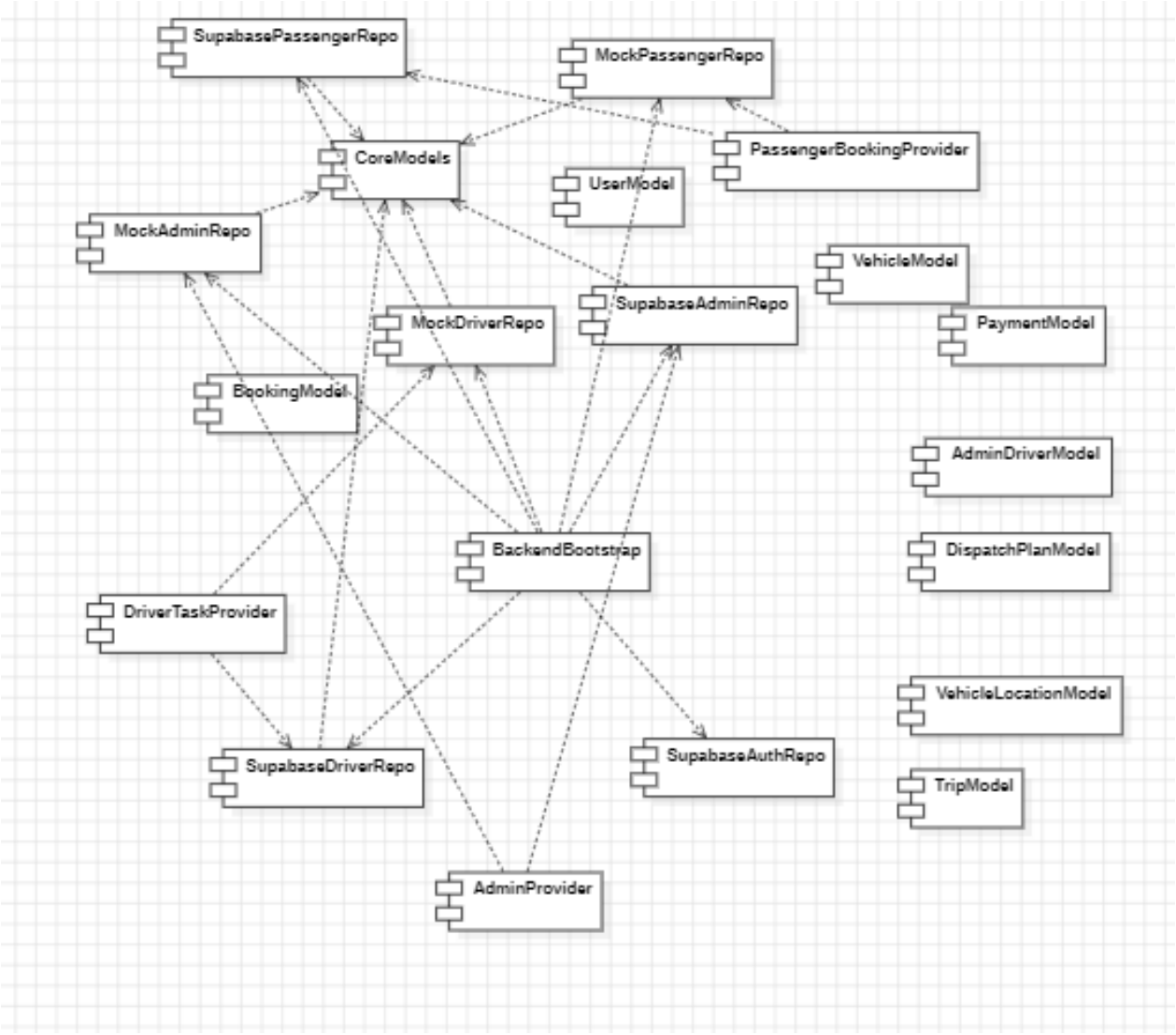


图 5.12: 目标数据层组件图：当前为独立 Mock 仓库和内存状态

5.4 StarUML 设计过程

1. 用例图：创建 Use Case Diagram，添加三种 Actor 及用例，使用 Association 关联角色与操作，Dependency 标注 include/extend；乘客图只保留乘客相关操作。
2. 类图：按核心数据字典添加 Class、属性与类型，画关联与多重性；另图使用 Enumeration 记录业务状态。
3. 状态图：创建 Statechart Diagram，以 InitialState、State、FinalState 和 Transition 表达对象生命周期；预约、车次、司机任务分别建模。
4. 时序图：建立 Actor、页面、Controller、Timer 等 Lifeline，连接 Message、自调用并核对 sequenceNumber；支付服务生命线属于目标抽象。
5. 部署图：使用 Node 展示客户端、应用层及第三方服务，用通信路径标注 HTTPS、REST、WSS；区分现状与规划。
6. 组件图：按表现、状态、仓库、模型和入口分层，以 Dependency 表示模块依赖；当真

实后端接入后再校正图中的预备组件。

图片通过 File → Export Diagram → PNG 导出；菜单支持时可以批量导出。更新时保留既有文件名，并在本正文中修改对应解释，避免恢复独立建模说明文件。

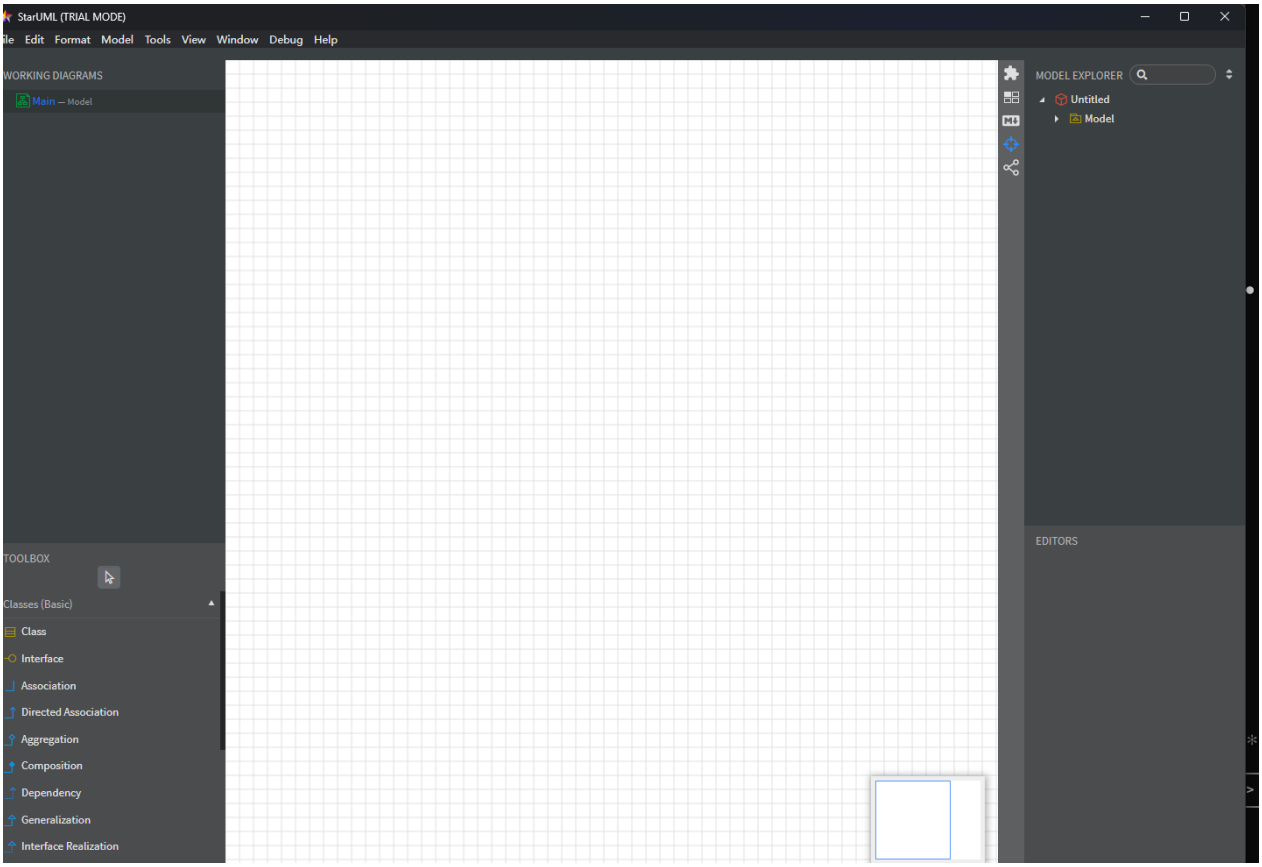


图 5.13: 历史 StarUML 软件主界面截图

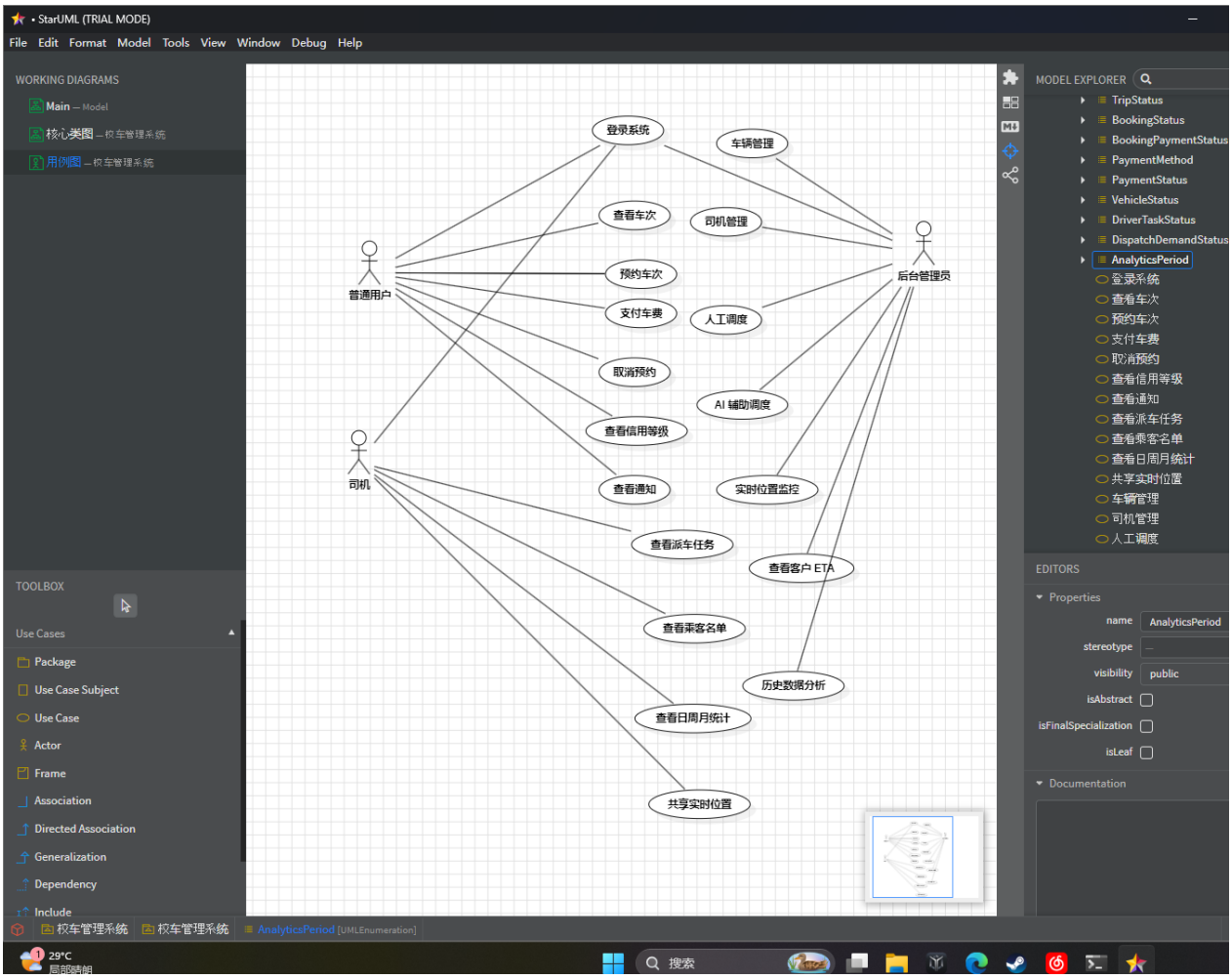


图 5.14: 历史用例图设计过程截图

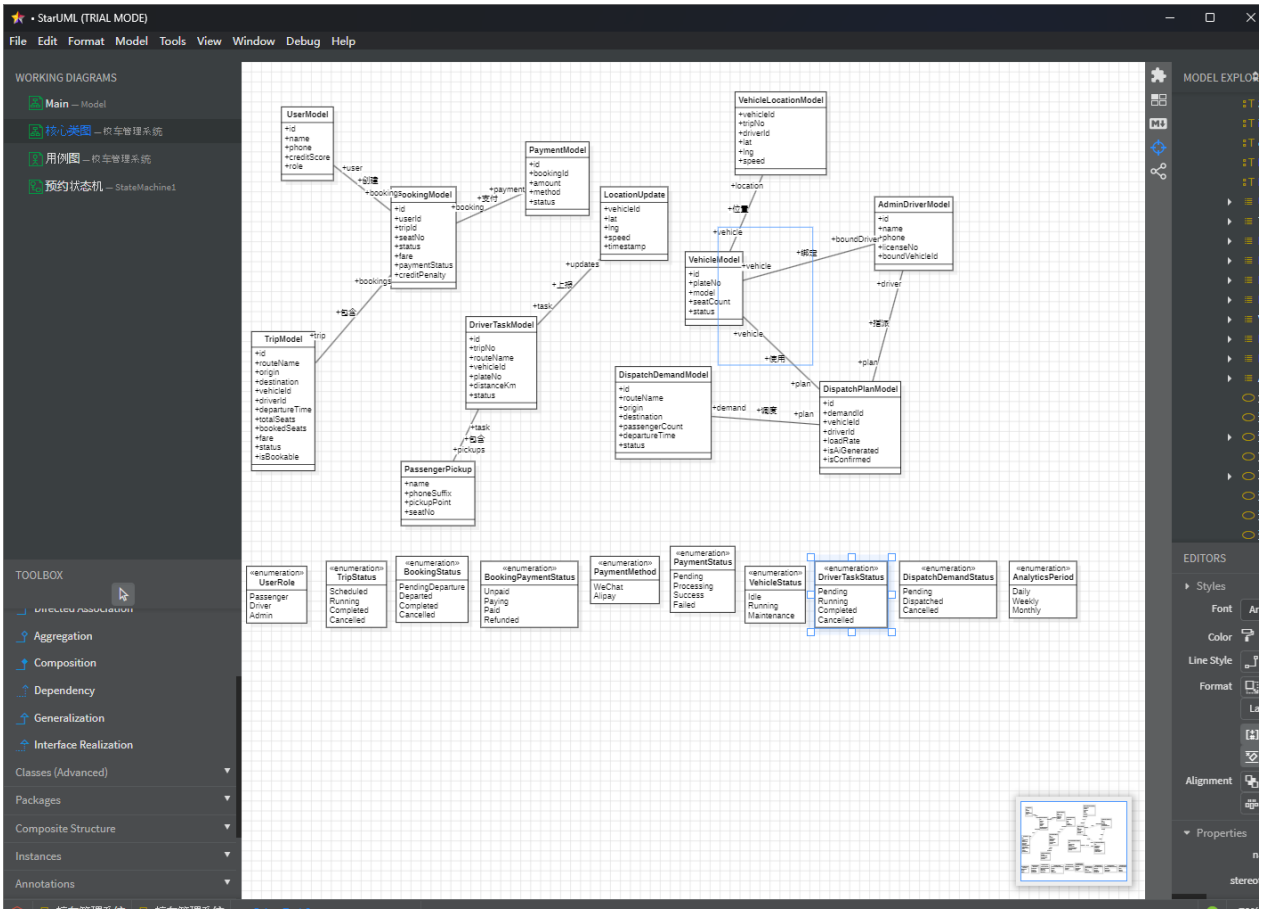


图 5.15: 历史领域类图设计过程截图

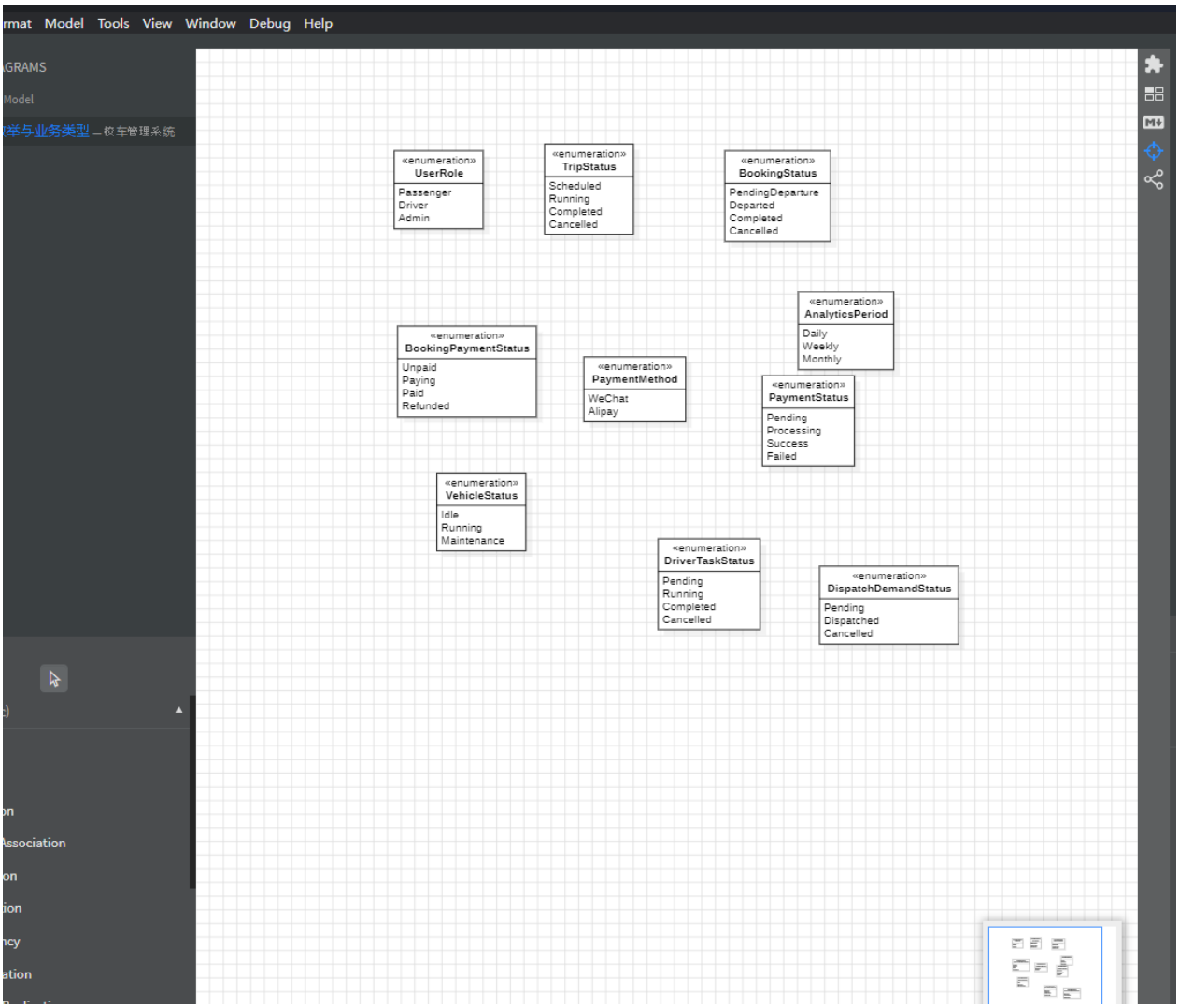


图 5.16: 历史枚举类图设计过程截图

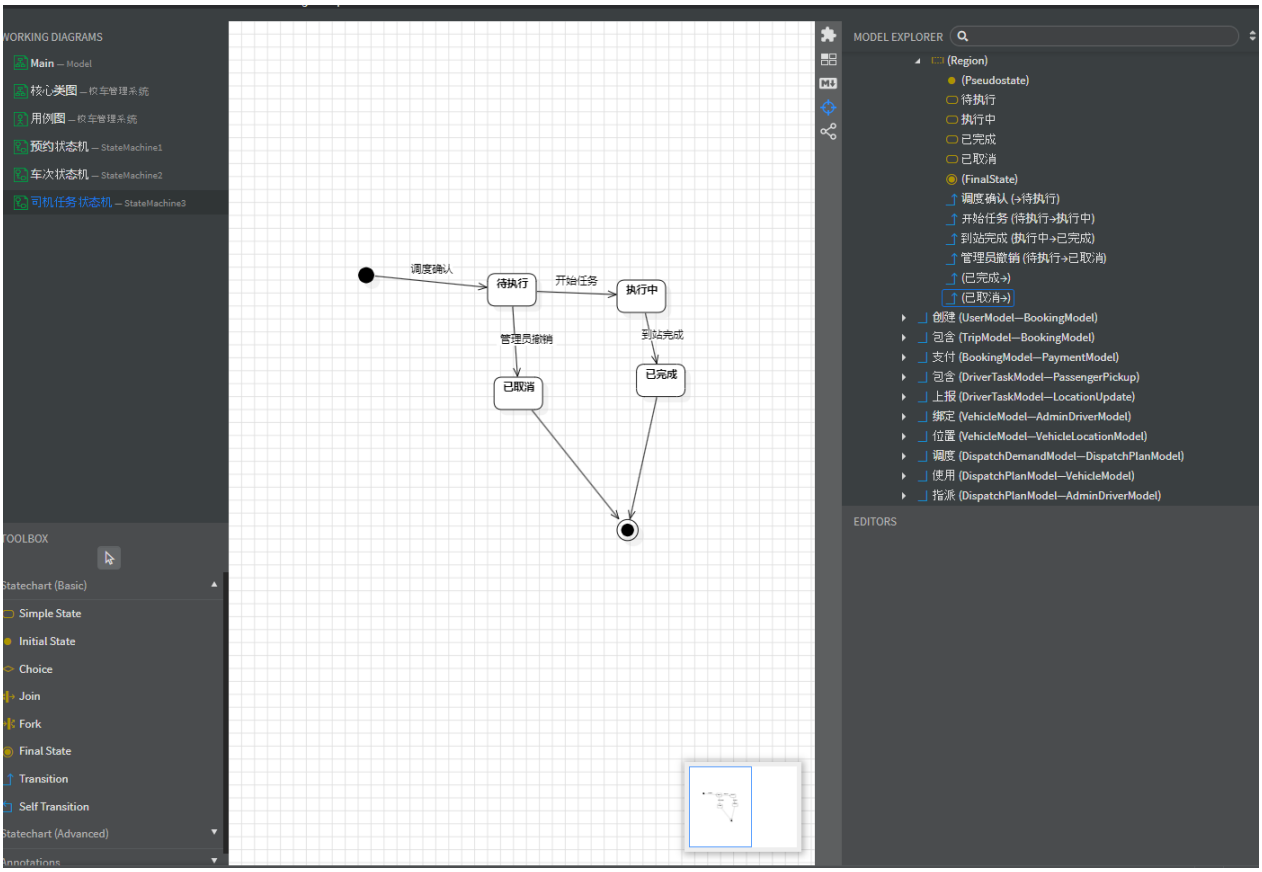


图 5.17: 历史状态图设计过程截图

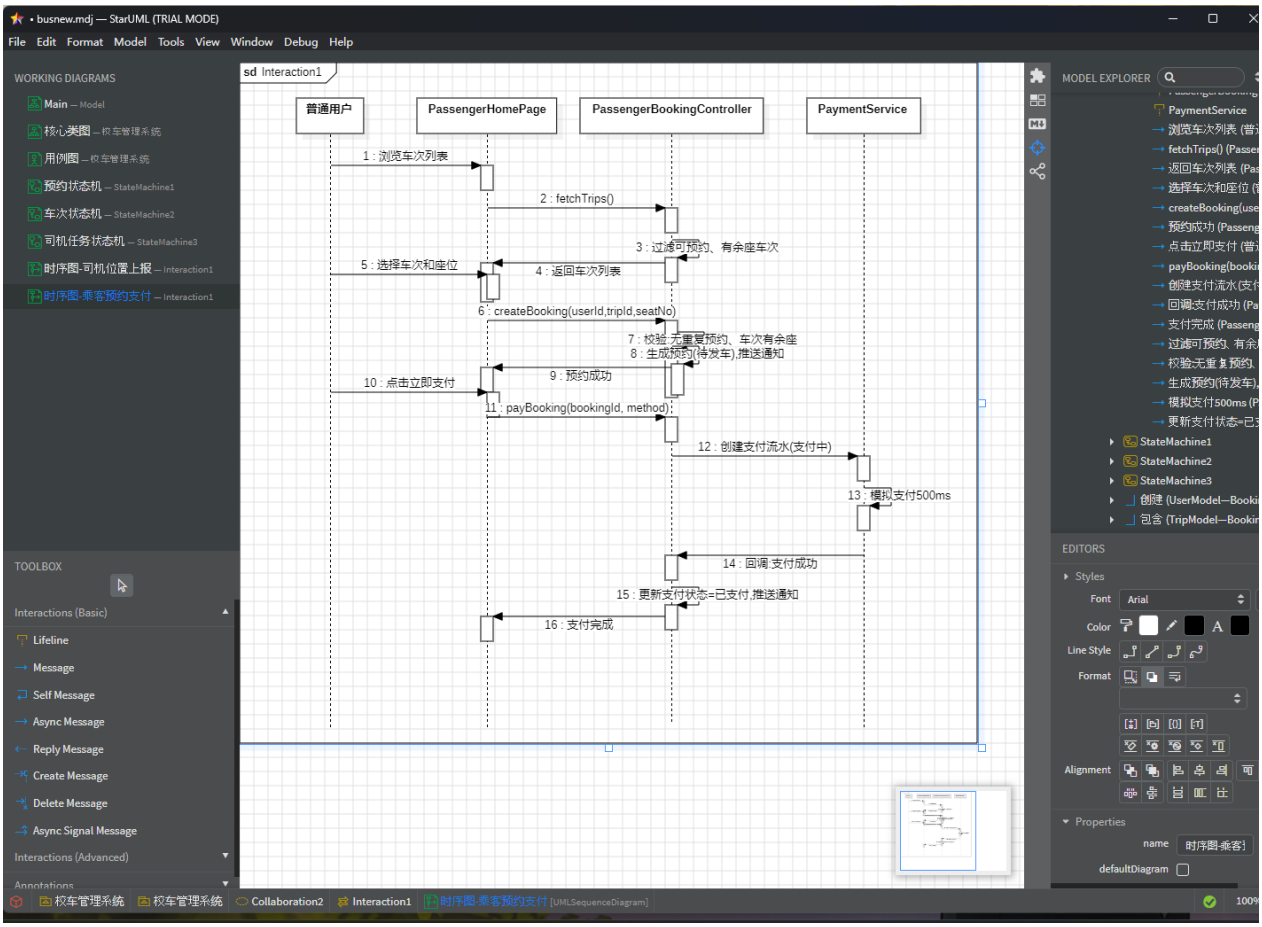


图 5.18: 历史时序图设计过程截图

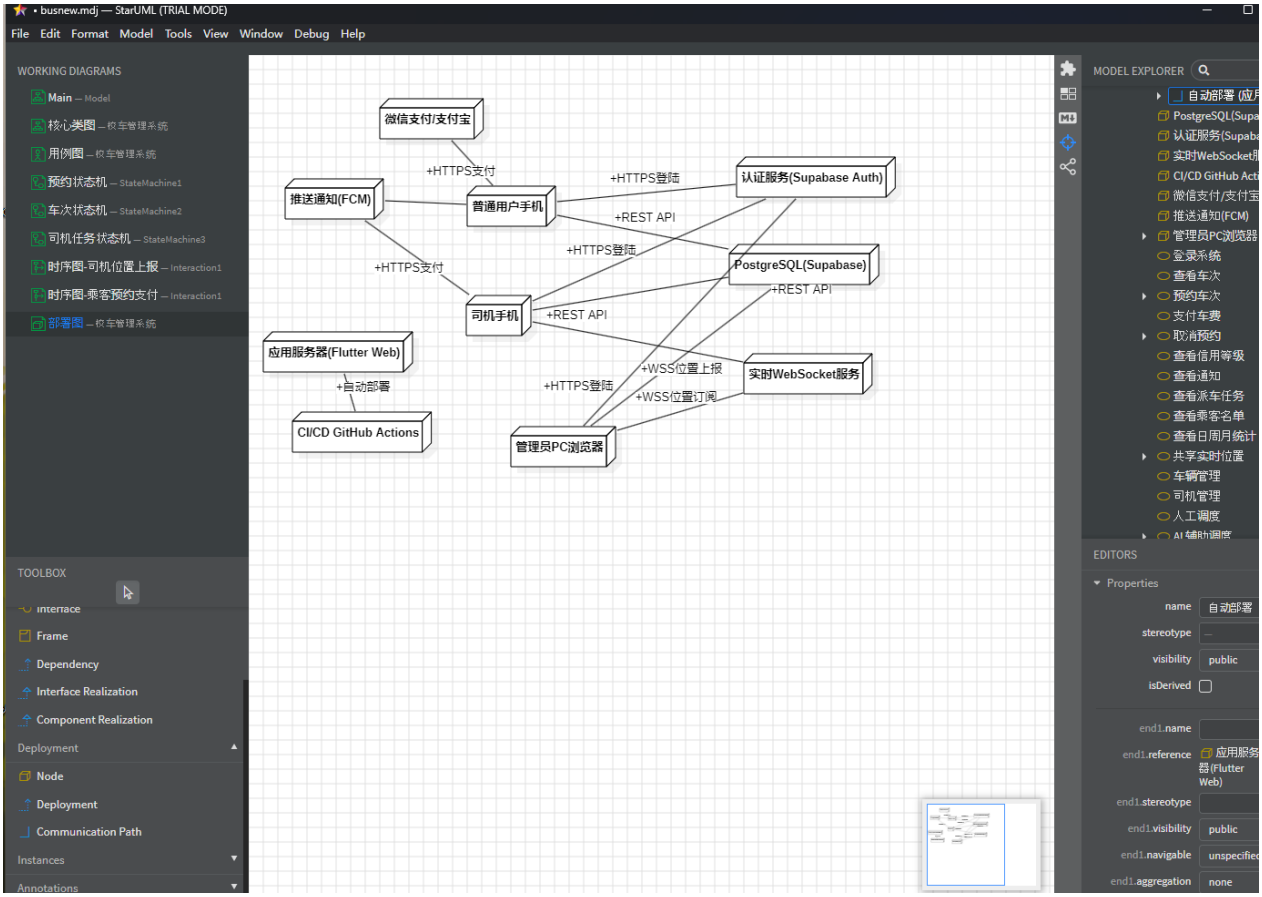


图 5.19: 历史部署图设计过程截图

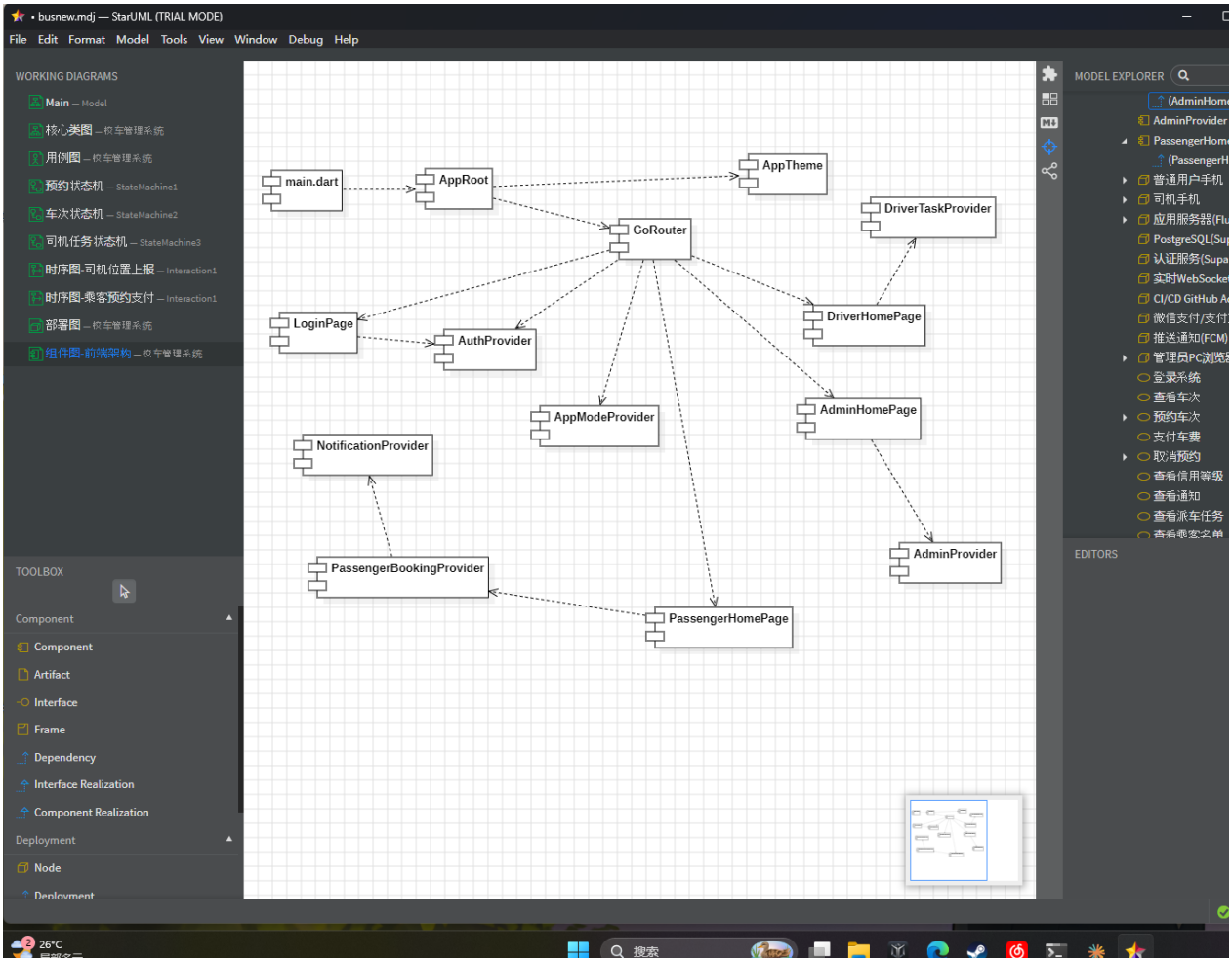


图 5.20: 历史组件图设计过程截图

第六章　平台工程与 Windows 宿主设计

课程工具标识： Flutter 平台工程、Win32 C++、CMake

6.1　平台边界

工程包含 Android、iOS、Web、Linux、macOS、Windows 的 Flutter 平台目录。业务入口仍是 Dart 的 `lib/main.dart`，平台 runner 只是宿主，不能把原 Windows 设计报告中的“考勤、设备告警、日志管理”等目标当作已实现业务。Android 的 Java / Kotlin 编译目标为 17，Gradle 配置采用 AGP 8.11.1、Kotlin 2.2.20；release 仍使用 debug 签名和示例 applicationId，不能直接称为正式上架配置。Linux runner 使用 GTK 3，其 CMake 安装后的 bundle 才包含完整资源，不能运行 intermediates_do_not_run 中的未打包程序。iOS 启动图可替换 `ios/Runner/Assets.xcassets/LaunchImage.imageset/` 中的图片，或在 macOS 打开 `ios/Runner.xcworkspace` 后在 Xcode 的 Runner/Assets.xcassets 修改。这一说明整合自平台模板 README。

6.2　Windows 入口、窗口和生命周期

文件 / 类	责任与关键方法
main.cpp / wWinMain	附加控制台、初始化 COM、配置 data 目录 DartProject、解析入口参数、创建 1280×720 的 school_bus_app 窗口、运行消息循环、反初始化 COM
Win32Window	封装 HWND、子窗口、Point、Size；Create、Show、Destroy、SetChildContent、GetClientArea、MessageHandler、UpdateTheme
WindowClassRegistrar	注册和注销原生窗口类，避免重复注册
FlutterWindow	继承 Win32Window，保存 DartProject，使用 unique_ptr 管理 FlutterViewController

`FlutterWindow.OnCreate`	创建并检查引擎和视图、RegisterPlugins、挂载子视图、首帧显示和 ForceRedraw
`FlutterWindow.OnDestroy`	释放 Flutter 控制器，再进行父类销毁
`FlutterWindow.MessageHandler`	优先转发引擎与插件，再处理字体变化并交给父类
utils.h / utils.cpp	CreateAndAttachConsole、GetCommandLineArguments、Utf8FromUtf16
resource.h / Runner.rc	图标和版本资源，图标 ID 为 101
`generated_plugin_registrant`	Flutter 生成的插件注册入口，不手动编辑

启动顺序为 wWinMain → COM / 参数 → DartProject → FlutterWindow / Win32Window → FlutterViewController → 插件注册 → 子窗口挂载 → 首帧 → Win32 消息循环。退出时销毁视图、原生窗口与 COM。消息处理优先级为 Flutter 引擎和插件、自定义 FlutterWindow、Win32Window、默认窗口过程。

6.3 构建、容错与扩展

`windows/runner/CMakeLists.txt` 要求 CMake 3.14，创建 WIN32 可执行目标；链接 flutter、flutter_wrapper_app、dwmapi.lib；注入版本宏并定义 NOMINMAX；依赖 `flutter_assemble`。新增原生源码需加入 runner 源文件列表；应用可执行名称由顶层 `windows/CMakeLists.txt` 的 BINARY_NAME 控制，不应只修改 runner 子目标名。

在 Windows、具备匹配 Flutter 要求的 Visual Studio C++ 桌面开发工具链时使用：

```
flutter run -d windows --dart-define=APP_MODE=admin
flutter build windows --release --dart-define=APP_MODE=admin
```

发行目录需整体分发，包含 EXE、Flutter DLL、插件和 data，不能只拷贝一个 EXE。当前 BINARY_NAME 为 school_bus_app，原设计文档 “Runner.exe” 只是泛称；应以实际产物为准。本次 Linux 环境未执行 Windows、Android 或 iOS 构建。

窗口创建失败和引擎无效时返回失败，智能指针管理视图控制器生命周期；字符编码转换检查空指针；DPI 和主题处理采用 Windows 平台实现。具体兼容性范围需要在目标 Windows 版本实测，而不是引用历史设计的版本声明作为保证。Platform Channel、串口/GPS/门禁、多窗口和本地日志属于后续扩展，当前没有相应业务通道。

第七章　源码使用、运行与演示

课程工具标识：Flutter / Dart、编辑器、Git

7.1　开发命令与账号

```
flutter pub get
flutter run --dart-define=APP_MODE=passenger
flutter run --dart-define=APP_MODE=driver
flutter run -d chrome --dart-define=APP_MODE=admin
```

无可用 Chrome 时，已配置 Web 开发环境可选择 `flutter run -d web-server --dart-define=APP_MODE=admin`，随后用浏览器访问命令给出的地址；不能把开发服务器当作真实业务后端。

入口	手机号	密码	用户
passenger	13800000001	123456	张同学，初始信用 96
driver	13800000002	123456	王师傅
admin	13800000003	123456	李管理员

这些账号只用于课程原型，不是真实凭据。页面快捷按钮先填充账号，再提交登录；账户角色需匹配当前入口。

7.2　高价值源码索引

路径	阅读目的
`lib/main.dart`、`lib/app.dart`	入口模式注入及 MaterialApp.router
`lib/core/router/app_router.dart`	登录鉴权和角色重定向

lib/core/providers/auth_provider.dart	认证状态、退出、信用变更
lib/features/auth/data/mock_auth_repository.dart	演示账号、密码及角色校验
lib/features/passenger/booking/providers/passenger_booking_provider.dart	余座、创建预约、取消、支付
lib/features/driver/tasks/providers/driver_task_provider.dart	activeTask、统计周期、位置 Timer
lib/features/admin/providers/admin_provider.dart	CRUD、绑定、调度确认、后台模拟位置
lib/core/providers/notification_provider.dart	本地通知状态
lib/features/*/presentation/	各端工作台与交互按钮
lib/shared/widgets/	入口切换、通用 Scaffold 与卡片

核心模型为手写 Dart 类和 copyWith，没有代码生成前置步骤。依赖安装不需要运行 build_runner，也不需要先启动 json-server 或 Supabase。

7.3 演示顺序

1. 乘客：快捷账号登录，预约大学城早班线 19 座，查看订单并模拟支付；另一次演示取消预约，核对信用 96→91 及通知。
2. 司机：查看乘客名单，切换日周月统计，开始 task-001，手动模拟一次位置记录，完成任务，检查按钮禁用及共享停止。
3. 后台：编辑 bus-002，新增车辆与司机，生成智能建议并确认，点击 bus-003 标记，查看固定客户 ETA，刷新模拟位置，再切换周统计。

演示三端时不能期待后台新增数据在另一个客户端出现；若刷新应用，内存修改会回到初始化样例。

第八章　测试计划、用例与报告

课程工具标识：Flutter Test、WidgetTester、ProviderContainer、测试执行记录

8.1　测试目标、范围和策略

测试覆盖三端主流程、业务拒绝路径、司机状态 getter / copyWith、Controller、模型与枚举。Widget 测试用 ProviderScope 注入指定入口，按 Key 定位按钮；业务单测用独立 ProviderContainer，完成后 dispose。当前自动化测试使用 Mock，不需要数据库、支付账号、GPS 设备或外部服务。

登录模拟 450ms，支付模拟 500ms；测试用 `tester.pump(Duration(...))` 推进假时钟。周期 Timer 及持续动画要显式控制，不能将 `pumpAndSettle` 当作等待所有后台任务的通用方法。滚动区域中的按钮先 `ensureVisible` 或 `scrollUntilVisible`；当前快捷登录 Key 是 `quick_${mode.value}_login`，不是旧测试样例的 mock 前缀。

8.2　乘客用例

编号	场景	操作及预期
TC-P-01	登录工作台	passenger 入口，账号 13800000001；显示预约车次和我的预约
TC-P-02	选座预约	trip-001 选择 19 座，确认；生成订单并显示成功提示
TC-P-03	取消与扣分	对新建待发车预约确认取消；扣 5 分，信用 96→91
TC-P-04	模拟支付	创建新订单，选择微信支付并确认；paid 和本地成功通知
TC-P-05	信用和通知	查看面板，预约 / 支付 / 取消后检查消息；部分由上述 Widget 合并覆盖

TC-P-06	错误角色入口	乘客账号登录 driver；抛出当前入口不匹配异常
TC-P-07	重复预约	同用户 trip-001 先预约 19 再预约 20；第二次拒绝
TC-P-08	非 pending 取消	booking-legacy-001/002；两者均提示只有待发车可取消

上述是需求场景，不等于八个独立自动化测试。乘客自动化测试分别位于 `widget_test.dart` 和 `business_rule_test.dart`；筛选切换、满座、弱网、真实支付和移动端兼容性仍需补充专项验证。

8.3 司机专项用例和测试数据

测试组	实际项数	验证点
登录与工作台	7	标题、司机姓名、任务和乘客数量、初始位置状态、路线、暂无上报
统计看板	4	默认日数据、周切换、月切换、准点率进度条
任务列表	8	路线与车次号、起终点、车牌、人次、里程、标签、名单、上车点
开始任务与上报	6	开始任务、首条位置、手动上报、坐标变化、车辆 ID、速度
完成任务	3	完成状态、停止上报、提示消息
边界情况	5	无任务时按钮禁用、单 active 任务、已完成不可操作、可开始下一任务
State 单元测试	9	任务数、乘客数、activeTask、统计选择、最新位置、copyWith 和清空状态
Controller 单元测试	7	启动、重复启动、不存在任务、完成、空任务完成、周期、上报数量
模型与枚举	8	任务 copyWith、三种状态、乘客、位置、统计、结果字段
合计	57	Widget 33 项，单元测试 24 项

原司机报告虽然总数 57 正确，但“开始任务与位置上报 7 个”“Widget 34 个”与明细不符，本表已按实际测试源码更正。

任务	路线	起止站点	乘客	里程
task-001	大学城早班线	南门学生公寓 → 综合教学楼	3	8.6km
task-002	科技园通勤线	图书馆广场 → 科技园东门	4	12.4km
task-003	晚间返校线	附属医院站 → 北区宿舍	2	15.8km

司机为王师傅，车辆 bus-001，车牌校 A · 1028；车次号依次为 D-20260501-01、02、03。模拟统计：日 4 趟 / 38 人 / 42.6km / 98%；周 24 趟 / 236 人 / 286.4km / 95%；月 98 趟 / 1034 人 / 1198.5km / 92%。

8.4 后台与负向业务用例

编号	场景	预期或验证点
TC-A-01	管理员登录	后台工作台、车辆与司机面板
TC-A-02	车辆编辑和新增	编辑 bus-002 车牌为校 A · 2040；新增校 A · 5098 显示在列表
TC-A-03	司机新增	姓名、手机、驾照保存后显示孙师傅
TC-A-04	建议生成与确认	显示 AI 推荐，确认后显示已确认
TC-A-05	车辆监控	点 bus-003；显示校 A · 3099、钱师傅和位置详情
TC-A-06	ETA 与位置刷新	固定预计 8 分钟样例；手动模拟刷新仍正常展示
TC-A-07	运营统计	切换周，显示样例人次 1298 及路线排行
TC-N-01	删除运行车辆	bus-001 删除失败：运行中车辆不可删除
TC-N-02	车辆绑定冲突	新司机尝试绑定 bus-002，失败：车辆已绑定其他司机
TC-N-03	人工调度容量不足	注入 10 座车辆及 36 人需求，失败：车辆座位数不足
TC-N-04	无可用智能车辆	注入空车辆 / 司机列表，返回没有可用于调度的空闲车辆和司机

后台主流程合并于一个 Widget 测试，并非七项独立自动化测试。七个负向自动化测试包含角色错误、重复预约、非法取消以及上表四项后台拒绝规则；测试仓库覆盖不足和无车数据通过 override 注入。

8.5 可复现的测试代码和命令

```
final container = ProviderContainer();
addTearDown(container.dispose);
final controller = container.read(driverTaskProvider.notifier);
expect(controller.startTask('task-001').success, isTrue);
expect(container.read(driverTaskProvider).activeTaskId, 'task-001');
expect(controller.completeActiveTask().success, isTrue);
expect(container.read(driverTaskProvider).activeTaskId, isNull);
```

```
flutter test
flutter test test/driver_test.dart
flutter test test/business_rule_test.dart
flutter test test/widget_test.dart --plain-name \
  'passenger can pay booking and receive notification'
# Optional coverage collection; no existing 80% claim is assumed:
flutter test --coverage
```

8.6 本次执行结果

2026-10-10 在 Linux、Flutter 3.47.5 stable、Dart 3.13.4 下执行源码验证。新 SDK 安装依赖时自动更新了六项测试/传递依赖和分析配置；这些与文档任务无关的改写不纳入交付，验证证据对应本机解析后的依赖环境，不宣称原锁文件在所有 SDK 上无需调整。

执行内容	本次结果
`dart format --output=none --set-exit-if-changed lib test`	退出码 0，无需改写源码格式
`flutter analyze`	退出码 0
`flutter test --reporter expanded`	退出码 0，69 项全部通过
`flutter build web --release --base-href /BUS-FLUTTER/ --dart-define=APP_MODE=admin`	退出码 0，生成后台 Web 构建

69 项组成：`driver_test.dart` 57 项、`widget_test.dart` 5 项、`business_rule_test.dart` 7 项。源码未因文档整合而变更。未执行真实网络后端、真实付款、真机 GPS、生产压测、安全测试、Windows 或移动平台发布测试。

8.7 历史测试结论与证据边界

原司机报告记录 2026-06-12、Windows 11、57 项通过、约 5 秒。原乘客报告日期 2026-06-10，并引用 2026-05-28 的“12 个场景通过”旧记录；它不是目前完整测试数。原后台手工报告环境为 Windows 11、Edge、Flutter 3.44.1 / Dart 3.12.1、Web Server、Mock，记载核心手工流程通过且无阻断缺陷；本次不把该结论作为新一轮浏览器手测。

原后台报告另含 JUnit 5 / Maven / Java UserTest 示例及截图，仓库不存在 pom.xml、对应 Java 模型或该测试源。这五个测试的创建用户、信用更新、余额更新、创建预约、取消预约仅作为历史补充演示，不计入 Flutter 69 项，也不说明 Flutter 已具有余额业务。

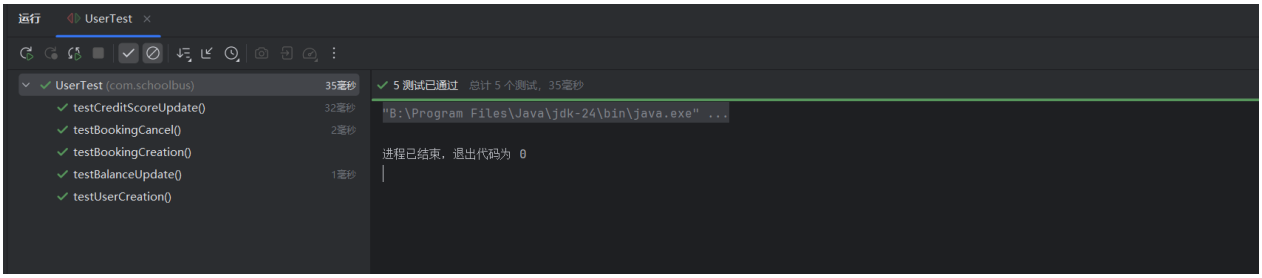


图 8.1: 历史 Java/JUnit 补充演示截图：不属于当前 Flutter 自动化测试

以下为原后台报告中的手工截图，时间和数据来自原材料，仅用于保留界面流程证据：



图 8.2: 历史后台手测：新增车辆对话框



图 8.3: 历史后台手测：新增车辆结果



图 8.4: 历史后台手测：车辆编辑对话框



图 8.5: 历史后台手测：车辆编辑结果



图 8.6: 历史后台手测：司机新增与绑定结果



图 8.7: 历史后台手测：智能建议生成、人工确认入口

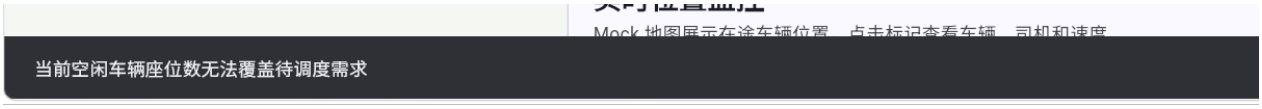


图 8.8: 历史后台手测：调度条件不足的提示

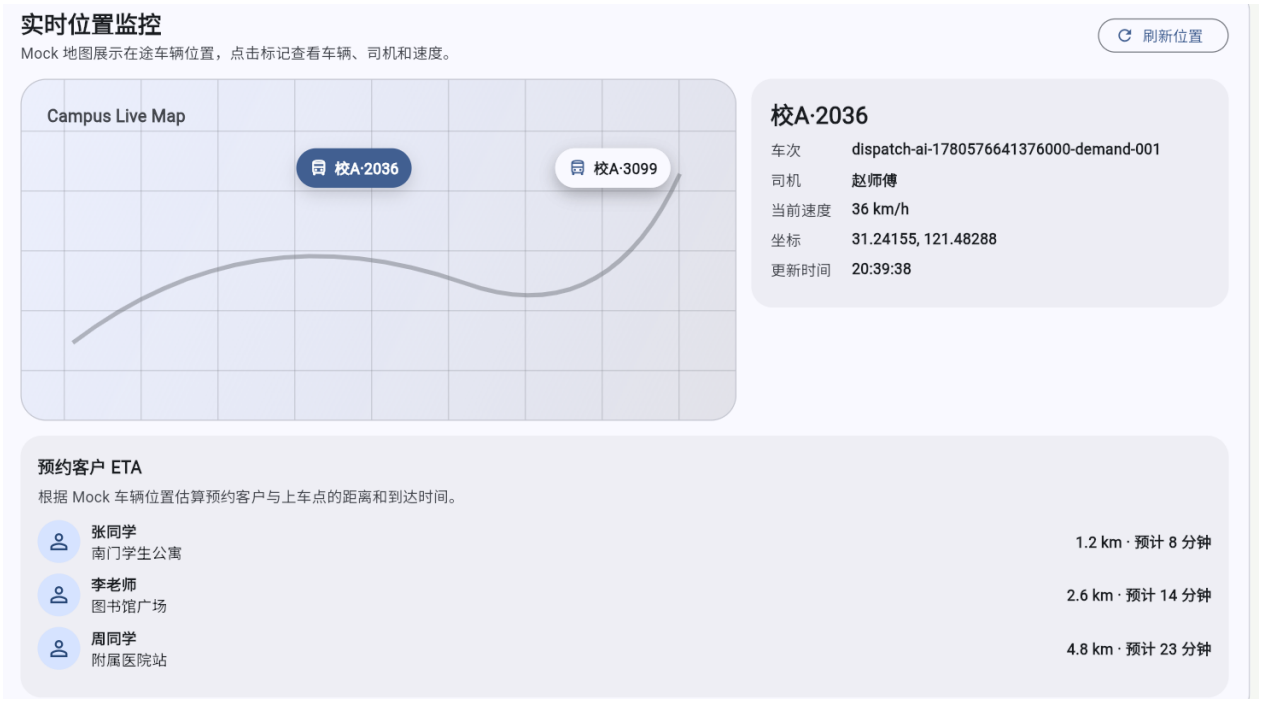


图 8.9: 历史后台手测：自绘位置面板与固定客户 ETA



图 8.10: 历史后台手测：位置变化局部截图

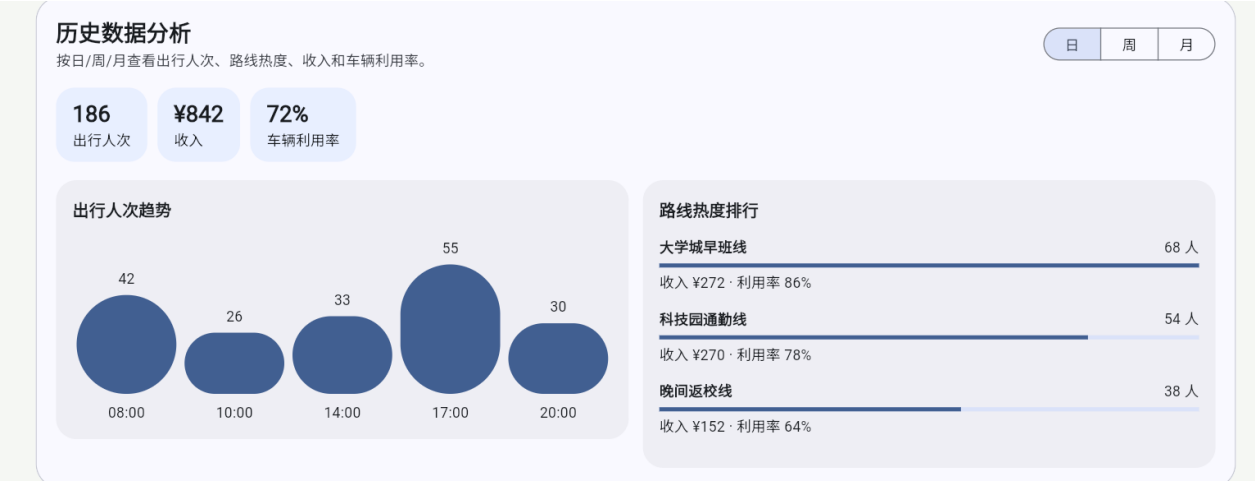


图 8.11: 历史后台手测：运营分析界面之一

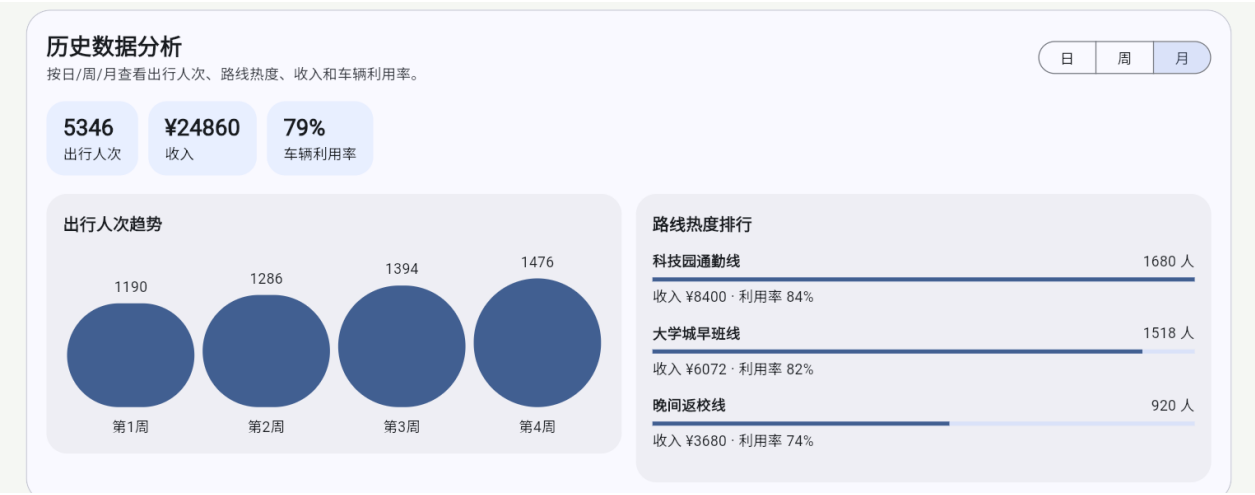


图 8.12: 历史后台手测：运营分析界面之二

8.8 剩余测试风险

需补充边界座位与满座、统计筛选、长时间 Timer 内存增长、支付中取消与重复支付、错误账户归属、资源编辑后的调度建议失效、确认同需求多个方案等场景。生产版本需加入跨客户端集成测试、网络异常恢复、数据库权限、真实支付回调、性能、国际化、无障碍及移动端兼容性。通过率不等于覆盖率，更不等于生产可靠性结论。

第九章　配置管理、CI/CD 与文档交付

课程工具标识：Git、GitHub、GitHub Actions、pubspec.lock、SQL 迁移

9.1　配置项与版本基线

配置项	当前路径与管理方式
源码和自动化测试	lib/、test/，Git 跟踪
依赖和静态分析	pubspec.yaml、pubspec.lock、analysis_options.yaml
平台工程	android/、ios/、web/、linux/、macos/、windows/
CI/CD	.github/workflows/flutter-ci-cd.yml
预备数据库	supabase/migrations/、supabase/seed/
唯一文档正文	main.tex；历史图像及原始证据在 report-assets/
Overleaf 交付	overleaf.zip，由 tools/build_overleaf.py 重建
入口与代理约定	README.md 只保留链接；AGENTS.md 仅记录代理工作约定

build/、.dart_tool/ 等本地产物不纳入版本控制。当前 `.idea/` 被忽略，`.vscode/` 忽略规则是注释，不应沿用原配置报告“所有 IDE 配置都已忽略”的笼统说法。真实支付密钥、服务端密钥和凭据不应写入公开文档；当前 App 并未读取原报告中列出的 Supabase 环境参数。

历史团队约定以 main 为课程主线，feature/*、fix/*、docs/* 为建议分支；提交类型 feat、fix、docs、test、refactor、chore。原文 v0.1–v0.5 对应从骨架到综合功能的阶段命名，是历史版本方案，不保证仓库已有对应 tag。当前 manifest 版本为 1.0.0+1。

核心预约、支付、调度、位置流程变更应先更新本文件中的需求、设计与风险，再实现并补测试；测试通过后经评审合并，并记录新基线。需求、设计、测试和交付基线如今分别对应本文件章节，而不引用已移除的分散报告路径。

9.2 CI/CD 的实际执行顺序

工作流在 main 的 push、指向 main 的 PR 及手动触发时运行，Ubuntu 环境安装 stable Flutter，然后按下列顺序验证：

```
flutter pub get
dart format lib test
flutter analyze
flutter test
flutter build web --release --base-href /BUS-FLUTTER/ \
  --dart-define=APP_MODE=admin
```

CI 的 format 命令会改写文件，不是严格的“发现格式变化就失败”检查；本地只检查可用 `dart format --output=none --set-exit-if-changed lib test`。工作流并不传 `BACKEND=mock`，因为当前不存在 BACKEND 读取逻辑。

只有 push 到 main 时上传 `build/web` Pages artifact 并执行部署；PR 只验证。GitHub Pages 需选择 GitHub Actions 作为构建与部署来源；Web 子路径固定使用 `/BUS-FLUTTER/`。去掉 base-href 会使子路径静态资源加载错误。

9.3 预备 Supabase SQL 与初始化顺序

`supabase/migrations/202605110001_initial_schema.sql` 包含 profiles、vehicles、drivers、trips、bookings、payments、dispatch_demands、dispatch_plans、vehicle_locations、notifications，定义状态枚举、约束、索引、更新时间触发器、辅助权限函数和 RLS policy。活动预约的座位和用户车次组合有条件唯一索引；driver 的车辆绑定列 unique；profiles 与 Supabase `auth.users` 外键相连。数据库 payment_method 还有 manual，而当前 Dart 只含 wechat/alipay，未来接入需显式适配。

若专门验证数据库，在隔离开发 Supabase 实例中先执行迁移，随后创建三名 Auth 用户，最后执行种子文件：

```
13800000001@school-bus.local / 123456
13800000002@school-bus.local / 123456
13800000003@school-bus.local / 123456
supabase/seed/202605110001_seed_demo_data.sql
```

种子依赖这三名 Auth 用户的 ID，未先创建用户会缺少相应 profile。当前仓库没有 Supabase CLI 配置、客户端初始化或后端切换，不编造 `supabase start` / 自动 reset 的现成工作流。现有 RLS 允许用户修改某些自身 profile/payment 字段，不能据此认为服务端信用和付款状态已被安全保护；生产接入需专项权限审查与事务设计。

9.4 单一文档打包与维护

`main.tex` 为唯一正文，`report-assets/uml/` 为 UML 与过程图片，`plan/` 为原计划图，`admin/` 为后台历史截图，`evidence/` 为 StarUML、MPP、CSV 可编辑证据。ZIP 顶层直接包含 main.tex 与 .latexmkrc，没有外层项目目录，没有旧 Word/PDF/Markdown 报告，也不要求 Overleaf 访问仓库外部路径。

```
python3 tools/build_overleaf.py
python3 tools/build_overleaf.py --check
# When a local TeX Live / XeLaTeX toolchain is installed:
latexmk -xelatex -outdir=build/report main.tex
```

脚本仅使用 Python 标准库，检查所有图像已被正文引用、资源存在、正文没有拆分输入，并以固定 ZIP 元数据打包。check 模式验证 ZIP 文件列表、CRC 和每项内容与当前源文件一致。修改正文、图片或证据后需重建交付包，不能只改 ZIP 或恢复多份平行正文。

第十章　后续全栈技术方案

10.1　保留的原始选型及其状态

本章整合最初技术方案，作为从课程原型走向真实系统的路线图。初稿 Riverpod 2.x、GoRouter 14 等依赖版本已经不代表当前项目；正式新增依赖时应按实际 Flutter / Dart 版本重新评估，而不是直接复制旧 pubspec 清单。

层	原方案候选	后续接入工作
网络	Dio + Retrofit	定义仓库接口、错误映射、超时重试、鉴权、真实 API
本地存储	flutter_secure_storage、Hive	Token 安全存储、缓存、失效和账号隔离
实时通信	WebSocket / socket_io_client	鉴权、重连、订阅、消息校验和序号
定位与地图	geolocator、flutter_map + OSM / 高德	平台权限、坐标系、后台采集和可视化
支付	微信 / 支付宝 SDK，初稿提及 pay	核对官方能力；pay 不等于两种支付的现成后端方案
推送	firebase_messaging / FCM	平台配置、推送 token、服务端通知与隐私
后端	json-server 原型 / Supabase	持久化、认证、RLS、服务端业务事务
图表	fl_chart	替换演示图表并连接真实统计数据
数据生成	json_serializable / freezed	只有选择生成模型后才引入生成命令
调度辅助	本地规则 / OpenAI	保留人工确认、安全校验与可解释约束

选择 Supabase 时可使用预备 SQL，但必须实现 Flutter 客户端和真实数据仓库，不能

通过一个未被读取的 dart-define 开关声称接入完成。json-server 仅可作为另行搭建的接口样机，仓库当前也没有 db.json。

10.2 目标 REST API

以下是接口设计，不是可请求的现有服务。当前操作直接调用 Notifier。生产实现需定义请求字段、响应结构、错误码、角色权限、分页、幂等键及并发事务。

```
POST   /api/auth/login
POST   /api/auth/register
GET    /api/trips
POST   /api/bookings
DELETE /api/bookings/:id
GET    /api/bookings/mine
POST   /api/payments
GET    /api/driver/tasks
GET    /api/driver/stats
GET    /api/admin/vehicles
POST   /api/admin/vehicles
PUT    /api/admin/vehicles/:id
DELETE /api/admin/vehicles/:id
GET    /api/admin/dispatch/suggest
POST   /api/admin/dispatch/confirm
GET    /api/admin/analytics
WS     /ws/location
```

司机 CRUD、订单支付查询、回调、退款、通知与统计导出尚需补齐接口契约。模拟订单、任务、后台需求 ID 与数据库 UUID 的映射也要明确，不能直接把当前演示字符串当数据库主键。

10.3 实时位置、推送与 ETA

原方案设计司机每 5 秒发出 `driver:location`，包含 vehicleId、lat、lng、timestamp，后台订阅 `vehicles:locationUpdate`。Socket.IO 事件协议与裸 WebSocket 路由不是同一种接口约定，服务端与客户端需统一后再接入。

```
// Planned Socket.IO event shape; not existing application code.
socket.emit('driver:location', {
  'vehicleId': vehicleId,
  'lat': latitude,
  'lng': longitude,
```

```
  'timestamp': timestamp,
});
socket.on('vehicles:locationUpdate', handleLocationUpdate);
```

正式客户端发车后启动 GPS、完成后停止采集，并在断线时恢复正确订阅；位置数据应有授权、精度、时效、保留期限与异常处理。乘客定位必须取得明确同意。ETA 可先用 Haversine 直线距离和假设速度做估算，再替换为路网与交通预测；必须标注估算误差，不得把当前固定 8/14/23 分钟样例当定位计算结果。

10.4 支付、数据一致性与调度演进

真实预约需在服务器以事务保护余座与座位唯一性；真实支付必须由服务端签名下单、验签回调与对账，客户端只能发起和展示状态，不能自行把订单改为已付。取消应统一计算信用、退款与座位释放，涉及支付中的竞态时使用幂等机制。智能调度可从当前贪心规则演进到路线聚合、多车分拆、司机排班、车辆维护窗口及全局优化；每条推荐应说明容量、负载和约束，人工确认时再次校验资源可用性。如果使用 OpenAI，仅将其作为辅助，不应绕过确定性安全约束或上传未经批准的个人位置数据。

第十一章　工具证据、整合追踪与总结

11.1　课程工具证据索引

阶段	工具标识	当前证据
计划	Project 类工具、CSV、甘特图	第二章，plan 图像，evidence/project_plan.mpp 与 project_plan.csv
需求	StarUML、规格说明与追踪矩阵	第三章，用例 PNG，StarUML 原模型
设计	StarUML、架构与平台设计	第四至六章，12 张 UML 和 8 张过程截图
源码	Flutter/Dart、Riverpod、GoRouter	第七章，lib/、pubspec.yaml
测试	Flutter Test、WidgetTester、ProviderContainer	第八章，test/、历史后台截图、本次实测结果
配置	Git、锁文件、GitHub Actions、迁移 SQL	第九章，工作流、pubspec.lock、supabase/
整合交付	LaTeX、XeLaTeX、Python ZIP	main.tex、.latexmkrc、tools/build_overleaf.py、overleaf.zip

编辑器和原生建模工具的使用来自历史记录与截图，本文不声称本次重新在 Project 或 StarUML 中操作过所有模型。

11.2　原材料合并与更正记录

原材料类别	整合位置	去重 / 更正说明
根目录 README 与技术方案	概述、运行、扩展方案	旧框架版本及全栈服务为规划，不作为当前依赖
计划 Word 与 PDF	第二章	相同正文合并一次，提取三张图片；6 月 4 日与 25 日分别说明
需求 Markdown	第三至五章	合并模型字典、用例、状态、追踪；修正 UML 数量与枚举
Windows 设计 DOC 与 PDF	第六章	保留原生宿主设计，区分模板能力与未实现业务
源码说明 Markdown	第四、七章	去掉原作者本机绝对路径，以仓库相对路径解释
司机测试 Word 与 PDF	第八章	三部分合并；实际 57 项，修正分组数量与旧 Key
乘客测试 Word 与 PDF	第八章	保留场景与风险，去掉已接入 Supabase 的错误说法
后台测试 Word 与 PDF	第八章	提取 12 张截图，JUnit 仅作为历史补充，按 Flutter 源码说明实测
配置管理 PDF	第九章	保留版本基线；纠正 BACKEND、IDE 忽略规则和 CI 命令
工具标识、最终报告	本章及各阶段	不再引用缺失的拆分报告、PlantUML 文件或旧交付目录
StarUML 过程说明及图片	第五章	保留原模型、全部 20 张 PNG；说明早期设计与当前实现差异
平台启动图 README	第六章	说明融入平台章节，不再保留额外正文
MPP、CSV、MDJ	report-assets/evidence/	原始可编辑证据保留，不产生平行文档

Word 与其 PDF 导出版本内容相同的部分不重复收录；历史截图未经伪造或改写。原报告引用但仓库实际不存在的独立司机测试计划/样例、配置 Word、PlantUML 等文件，不再作为已交付证据列出。

11.3 最终交付与结论

唯一阅读正文为本综合报告；Overleaf 项目由 main.tex、编译配置、图像和可编辑计划/建模证据组成，导入后编译得到一份完整 PDF。源码、测试、平台工程和预备 SQL 仍作为工程资产保留，不被文档合并替代。

当前课程原型已具备三类工作台与被覆盖的核心交互，本次 Linux 验证的格式检查、静态分析、69 项自动化测试和后台 Web 构建均通过。后续正式上线需要真实认证、持久化、跨客户端数据一致性、实际支付与定位、权限审查以及生产级测试。将这些限制明确写入同一份报告，比保留多份互相冲突的“最终报告”更有利于评审、维护与后续开发。